

sirr — Küçük-Model Mimari Denetim Raporu

Alt başlık: Apple FoundationModels (~3B parametre, ~4096 token bağlam) küçük-model uyumu için mimari denetim ve iyileştirme yol haritası

Proje: sirr (repo/target: **ketum**) — tamamen cihaz-üstü, çevrimdışı Türkçe iOS asistanı **Tarih:** 19 Temmuz 2026 **Kapsam:** Kod tabanının tamamı (84 Swift dosyası, ~17.100 satır) — 9 mimari boyut **Not:** Bu denetim yalnızca inceleme + rapordur; **kod değiştirilmemiştir.**

0. Yöntem

Denetim, dosya sahipliğine göre değil **mimari boyuta göre** bölünmüş paralel bir çok-ajanlı iş akışıyla yürütüldü:

- **9 bağımsız denetçi ajan** eşzamanlı çalıştı; her biri bir boyutu (istem/ token bütçesi, niyet profili & araç bütçesi, araç şemaları, MCP köprüsü, beceri/guidance enjeksiyonu, yapısal çıktı ayrıştırma, toplu veri offload, hata/retry/iptal, değerlendirme & öz-test) yalnızca-okuma modunda inceledi.
- Her bulgu **dosya:satır** kanıtına ve somut **küçük-model etkisine** bağlandı.
- Bir **sentez ajanı** 9 boyutun yapısal bulgularını birleştirip çelişkileri çözdü, önerileri P0/P1/P2 olarak tek yol haritasında topladı.
- Toplam: **10 ajan, 0 hata, ~707.000 token, 161 araç çağırısı.**

Değerlendirme eksenini her boyutta sabit tutuldu: *token bütçesi, istem uzunluğu, araç sayısı/şema karmaşıklığı, çıktı ayrıştırma kırılganlığı, halüsinasyon yüzeyi, deterministik güvenceler.*

sirr Mimari Denetim Raporu — ~3B / 4096-Token FoundationModels Uyumu

1. Yönetici Özeti

sirr'in çekirdek felsefesi — "deterministik iskele kurup küçük modeli olabildiğince dar bir karar alanına hapsetmek" — küçük-model gerçekliğine doğru cevap veren, sınıfının üstünde bir tasarımdır: araç SETİ modelce değil kodca seçilir (ModelServisi.swift:527-556), token bütçesi tahminle değil gerçek **tokenCount** ile yönetilir (ModelServisi.swift:901-908), toplu veri modelden geçmeden VeriDeposu üzerinden taşınır (KetumAraci.swift:41), kod/HTML çıktısı deterministik olarak doğrulanır (KodMotoru.swift:96-98, SayfaDogrulayici.swift). Ancak denetim, tutarlı bir çatlak deseni ortaya koyuyor: **deterministik güvence bir katmanda kurulup bir sonraki katmanda modele/istme geri bırakılıyor**. En kritik üç kaldıraç: (1) sabit istem maliyeti belgelenenin ~4-5 katı (~2100-2400 token, 4096'nın yarısı) olduğu için konuşma penceresi daha başlamadan tükeniyor (Yonlendirici.swift:5 vs :25-64); (2) discriminator/biçim alanlarının gerçek enum yerine düz **String** olması, referansın turlar arası taşınmasının istme bırakılması ve uzak MCP yazmalarının retry-korumasının dışında kalması, küçük modelin en olası sapmalarında **sessiz veri kaybı ve çift dış yan etki** üretiyor (TakvimAraci.swift:30, BelgeOlusturAraci.swift:46-54, MCPAraci.swift:227-235); (3) modelin asıl kırılğan davranışını (dil çapısı, argüman doğruluğu, format uyumu) ölçen gerçek-model testleri hiçbir otomatik kapıya bağlı değil (Degerlendirme.swift:190-192) — yani yukarıdaki her düzeltme regresyona karşı korumasız. Kısaca: omurga sağlam, kırılğanlık omurganın uçlarında.

2. Mimari Genel Değerlendirme

Tasarımın temel iddiası — "küçük model halüsinasyona ve kırılğan araç çağrısına yatkın olduğu için onu deterministik kabuklarla çevrele" — tutarlı biçimde uygulanmış ve çoğu yerde **fiilen işliyor**. Güçlü olduğu yer, kararın modelden ALINDIĞI her nokta: araç seti seçimi (ModelServisi.swift:527-556), araç sayısı tavanı (6-8, çeviriden önce uygulanıyor, ModelServisi.swift:1037), sonraki-tur sinyalinin model metninden değil chip ikonundan okunması (ModelServisi.swift:590-593), onay kapısının kodda tutulup modele bayrak sızdırılmaması (AracYurutucu.swift:50-53), ve toplu verinin **modeleDonen/hamCikti** ayrımıyla modelden yapısal olarak gizlenmesi (KetumAraci.swift:41). Bunlar küçük modelin "iyi niyetine" değil koda dayanan güvenceler.

Çatlak, kararın modele ya da istme GERİ BIRAKILDIĞI yerlerde açılıyor ve şaşırtıcı biçimde tekrar eden bir desen izliyor:

- **Deterministik altyapı var ama son adımda metne düşülüyor:** araç

başarısızlığı yürütücüde bilinebilirken modele İngilizce sentinel dizge olarak dönüyor (KetumAraci.swift:51), retry güvenliği tek **String** durum bildirimine (**.yazildi**) güveniyor (AracYurutucu.swift:163), tablo yapısı katı tek biçime bağlı (Tablo.swift:63-65).

- **Katmanlar birbirini tanımıyor:** beceri enjeksiyonu aktif profilin araç setinden habersiz (BeceriDeposu.swift:64), iki paralel anahtar-kelime sistemi (profil izleri vs beceri tetikleri) elle senkronize ediliyor (ModelServisi.swift:596-657 vs **.md tetikler**).
- **Mekanizma kurulmuş ama üretime/koruma altına bağlanmamış:** çekirdek beceri kesme sessiz (BeceriDeposu.swift:78-85), ref-miss sessiz başarı (BelgeOlusturAraci.swift:78-87), gerçek-model eval'i eşiksiz dosyaya yazıyor (Degerlendirme.swift:190-192).

Yani mimari, küçük-model için doğru içgüdülerle tasarlanmış ama içgüdünün SON MİLİ deterministik olarak kapatılmadığı için, tam da en kırılgan modelin en sık düşeceği kenar durumlarda güvenceler devreden çıkıyor.

3. Boyut Boyut Bulgular

3.1 İstem inşası & token bütçesi

Mevcut durum: Progressive disclosure gerçek (beceriler talimata gömülüyor, tek beceri bir kez iliştiliyor — ModelServisi.swift:344-409), bütçe gerçek **tokenCount** ile %80 eşiğinde yönetiliyor (ModelServisi.swift:904), özet ayrı oturumda üretiliyor (ModelServisi.swift:928-930), ham araç çıktısı geçmişe taşınmıyor (ModelServisi.swift:942-958). Altyapı sınıfının üstünde. **En şiddetli zayıflıklar:**

- Sabit maliyet iddia edilenin ~4-5 katı: **talimatlarEN** "~150 token" diyor (Yonlendirici.swift:5) ama blok 2833 karakter $\approx$ ~650-700 token; üstüne non-EN dil çapası ~200 token (ModelServisi.swift:434-445) + 8 araç şeması ~1200-1500 token = bir Türk kullanıcı için **~2100-2400 token, 4096'nın ~%50-60'ı** sohbet başlamadan tükeniyor. Tasarım kararları (özetleme eşiği, korunan tur) yanlış varsayımla ayarlanmış olabilir.
- Profil-dışı kalıcı bloklar her profile ödeniyor: SEARCH RESULTS paragrafı (Yonlendirici.swift:44-50) ve belge-export akışı (Yonlendirici.swift:57-60) sabit talimatta, oysa gündelik profile web_arama aracı yok (ModelServisi.swift:280-290) — olmayan aracın varlık beklentisi halüsinasyon yüzeyi doğuruyor.

- Yanıt dili üç ayrı yerde pekiştiriliyor (Yonlendirici.swift:29-32 + ModelServisi.swift:434-445 + :404), tek kaygıya ~350+ token, her oturum kuruluşunda yeniden.

Öneri: Talimatı dil/araç-nötr kısa ÇEKİRDEK + profil-özgü ek olarak böl; çekirdeği ~250-300 token'a indir, ayrıntıyı beceri katmanına taşı; dil pekiştirmesini en yakın kanala (tur-başı [Reply in X]) indir.

3.2 Niyet profili & araç bütçesi

Mevcut durum: Araç seti deterministik, 6-8 tavan fiilen uygulanıyor, sonraki-tur sinyali çip ikonundan (halüsinasyona kapalı, ModelServisi.swift:587-593). Doğru omurga. **En şiddetli zayıflıklar:**

- İlk-eşleşen-kazanır çapraz-sinyali öldürüyor: "notlarımı excel yap" önce **gundelikIzleri**'ndeki "notlarım"a takılıp gündelik profile düşüyor (ModelServisi.swift:539), o profilde belge_olustur YOK (:289) → kullanıcının açık dosya isteği sessizce kayboluyor.
- Yanlış-profilde tur-içi kurtarma yok (ModelServisi.swift:527-556): seçici yanılırsa gereken araç oturumda hiç yok, model başarıyı uyduruyor — hiçbir deterministik güvence devrede değil.
- gündelik/belge tam 8 araçla tavanda; kodun kendi yorumu tavanın "zorlandığını" kabul ediyor (ModelServisi.swift:276-279). Nöbet/Kod seyrek ama her turda ayırım yükü ekliyor.
- Operasyon argümanları serbest **String** + bulanık **.contains** (bkz. 3.3).

Öneri: Tur-içi yanlış-profil kurtarma kapısı ekle (P0); profil seçimini sinyal-skorlamasına taşı; gündeliği 6 araca indirip Nöbet/Kod'u ayrı profile al; ekli belge kilidini güçlü belge-dışı sinyalde yumuşat.

3.3 Araç şemaları & açıklamaları (@Generable/@Guide)

Mevcut durum: Constrained decoding var, model serbest metin parse ettirmiyor (KetumAraci.swift:6-7); araçların çoğu tek zorunlu string (Hesap/Kişi/Arama/WebArama) — küçük model için ideal. MCP enum'ları gerçek constrained listeye çevriliyor (MCPAraci.swift:406-412). **En şiddetli zayıflıklar:**

- **Yerli discriminator/biçim alanları gerçek enum değil, düz **String** + bulanık eşleştirme:** `var eylem: String + arguments.eylem.lowercased().contains("ekle")` (TakvimAraci.swift:19-30); model İngilizce turda `eylem="add"` üretince "ekle" bulunmaz, kod SESSİZCE okumaya düşer — kullanıcı

etkinlik eklerken liste görür (HatirlaticiAraci.swift:185-186, ZamanAraci.swift:21 aynı desen). @Generable enum kullanılsa çerçeve geçerli case'e zorlar, bu sınıf tamamen kalkar.

- belge_olustur şeması: JSON string İÇİNE elle hizalı markdown tablo + **icerik/kaynakRef** karşılıklı dışlaması yalnız düzyazıda (BelgeOlusturAraci.swift:30-32) — bozuk Excel/boş belge riski.
- TR/EN dil karışımı: alan adları Türkçe, değerler Türkçe, açıklamalar İngilizce, tek enum içinde karışık ('metin' vs 'markdown', BelgeOlusturAraci.swift:24) — 3B modelin diller-arası eşlemede tökezlediği tam nokta.
- kod_calistir **dil** alanı ölü ama zorunlu decode slotu (KodCalistirAraci.swift:42-44, :70).

Öneri: Discriminator/biçim alanlarını @Generable enum'a çevir (P0); ölü **dil** alanını kaldır; enum'a geçince dil karışımı kökten çözülür; MCP per-property açıklamalarına da uzunluk tavanı koy (MCPAraci.swift:389 şu an ham geçiyor).

3.4 MCP köprüsü & dinamik araç yükü

Mevcut durum: Dört katmanlı tavan çeviriden önce uygulanıyor (200→6, ModelServisi.swift:1037), tek bağlantı seçiliyor, tanım on-device özetlenip önbellekleniyor (BaglantiServisi.swift:309-327), derinlik sınırı 4 + anyOf/\$ref reddi (MCPAraci.swift:327,368). Savunma iyi kurulmuş. **En şiddetli zayıflıklar:**

- **Şema GENİŞLİĞİ hiç sınırlanmıyor, yalnız derinlik:** derinlik-1 ama 40 alanlı tek **object** sınırı geçip olduğu gibi modele giriyor (MCPAraci.swift:380-393); "6 araç" tavanı olsa da her araç genişlikte token bombası olabilir, 3B model 40 alanı doğru dolduramaz.
- 6-arac bütçesi alakaya değil sunucunun **tools/list** sırasına göre kör dolduruluyor (ModelServisi.swift:1037): gereken araç 7. sıradaysa modele HİÇ görünmez — en pahalı kaynak rastgele harcanıyor.
- Araç adı çakışması çözülüyor; kod satır 129'daki "önek almış olabilir" vaadini tutmuyor (MCPAraci.swift:276-282) — "get-user"+"get_user" ikisi de "get_user"a inip profili düşürebilir.
- İçeride aktarmada tavan uygulanmıyor: 200 araca kadar hepsi tek tek on-device özetletiliyor (BaglantiServisi.swift:274-280), yalnız 6'sı kullanılıyor — dakikalarca arka plan çıkarımı.
- Araç çıktısına gömülü İngilizce imperative'ler (bkz. kesişen tema, 3.8).

Öneri: Şema genişliği için alan/düğüm bütçesi ekle; 6-araç yuvasını hafif metin-eşleşmesi + **sonKullanım** sinyaliyle alakaya göre doldur; ad çakışmasını dedup+önekle çöz; özetlemeyi de tavan+alaka ile sınırla.

3.5 Beceri/guidance enjeksiyonu

Mevcut durum: Tek beceri, bir kez, sistem talimatına değil turun istemine ve sorunun hemen önüne enjekte ediliyor (ModelServisi.swift:382-389) — son-blok ağırlığından yararlanan doğru tercih; uzunluk-ağırlıklı puanlama gerçek belirsizliği çözüyor (BeceriDeposu.swift:64-74). **En şiddetli zayıflıklar:**

- **700 karakter kör kesme, en değerli iki parçayı atıyor (KRİTİK):** BeceriDeposu.swift:78-85 gövdeyi sondan kesiyor; ölçüm: kod.md 1069→690'da **## Example** (kod_calistir örneği) VE **## Rules** ("Never claim you ran code without a successful tool call") tamamen düşüyor; web-sayfa.md 1248→683 örnek+kurallar düşüyor; belge-olustur.md "NEVER compute totals yourself" =SUM kuralı düşüyor. Yani beceri, tam da çözmeye çalıştığı halüsinasyonu (uydurulmuş sonuç, kafadan toplam) üretecek hale geliyor.
- Alt-dizge eşleşmesi (kelime sınırı yok): 'bul' ∈ 'bulutlu' → hava sorusu arama becerisini kazanıyor, 'oku' ∈ 'dokun' → belge-oku (BeceriDeposu.swift:68); yanlış kılavuz küçük modeli yanlış davranışa kilitliyor.
- Enjeksiyon aktif profilin araç setinden habersiz: 'python ile dolar hesapla' → profil=.arama (kod_calistir YOK) ama skill "Call kod_calistir" diyor (ModelServisi.swift:320-325 vs BeceriDeposu.swift:64) — şema-dışı çağrı.
- İki paralel elle-senkron liste (profil izleri vs .md tetikler) kayabiliyor.

Öneri: Kör kesmeyi "çekirdek-önce" modele çevir — örnek+kritik kural gövdenin başına, enjeksiyon yalnız çekirdeği alsın (P0); tetik eşleşmesini kelime-sınırlı yap, kısa tetikleri exact-only (P0); beceriyi profil araç setiyle çaprazla; tek tetik kaynağından türet.

3.6 Yapısal çıktı & ayrıştırma kırılğanlığı

Mevcut durum: Akış sırasında yarım tablo render edilmiyor (KetumYaniti.swift:147-152), kod sandbox deterministik (KodMotoru.swift:96-98, 2 çalıştırma tavanı), HTML ekran-dışı doğrulanıyor (BelgeOlusturAraci.swift:65-77). Güçlü deterministik güvenceler. **En şiddetli zayıflıklar:**

- Tablo algılama kapısı çok katı: dış-pipe + ayraç satırı + veri satırı üçlüsünü şart koşuyor (Tablo.swift:63-65); 3B model sık sık dış-pipe'sız ('Kahve | Fiyat') ya da ayraçsız üretir → kapı açılmaz, kullanıcı ham `| a | b |` görür. Format baskısı (Yonlendirici.swift:40) "satırları bas" derken parser tek katı biçimi kabul ediyor — talimat ile tolerans çelişiyor.
- **Excel yolunda bozuk tablo → tek-sütunlu çöp xlsx (sessiz yapı kaybı):** `markdownDan` nil dönerse ham metin tek sütuna dökülüyor (BelgeDuzenleAraci.swift:46-50, ExcelMotor.swift:25-28), kullanıcı "düzenlendi" çipi görüp bozuk dosya buluyor — yanlış başarı sinyali.
- Yarım/bozuk tablo sessiz KAYIP: `markdownDan` nil ise toplanan pipe satırları hiçbir yere eklenmeden atlanıyor, else dalı yok (KetumYaniti.swift:195-202).
- `belge_olustur` yalnız ilk tabloyu alır, kalanı düşürür (BelgeOlusturAraci.swift:49-53).
- Bozuk-çıktı testi yok; parser sözleşmesi yalnız kusursuz markdown ile test ediliyor (OtoTest.swift:87-94).

Öneri: Toleranslı tek parser (ayraç opsiyonel, dış-pipe opsiyonel) + nil dönerse pipe satırlarını asla düşürme, `.metin` olarak geri ekle (P0); Excel yolunda bozuk tabloyu HTML deseni gibi düzeltme döngüsüne sok; çoklu tablo+çevre metnini koru; bozuk-çıktı test matrisi ekle.

3.7 Toplu veri offload & 4096 bypass (VeriDeposu + kaynakRef)

Mevcut durum: Kanal ayrımı deterministik ve tek noktada (KetumAraci.swift:41); referans üretici monotonik, `temizle()` sayacı sıfırlamıyor (VeriDeposu.swift:40-48) — eski ref çarpması yapısal olarak kapalı; `al()` toleranslı (tırnak/boşluk kırıyor). Sağlam kurulmuş. **En şiddetli zayıflıklar:**

- **kaynakRef çözülemezse SESSİZ başarı (KRİTİK):** `al(ref)` nil dönerse dal atlanıyor, `govde = arguments.icerik` (model ref kullanınca içeriği boş bırakması söylenmiş) ile devam edip her hâlükârdâ `file_created` dönüyor (BelgeOlusturAraci.swift:46-54, 78-87). Kullanıcı BOŞ/eksik Excel alıp "Hazır" duyuyor — teşhisi zor veri kaybı.
- Referans etiketi tutarsız: kaynak araçlar modele `data_ref=` gösteriyor (TakvimAraci.swift:73) ama argüman adı `kaynakRef`, MCP ise `kaynakRef=` diyor (BaglantiServisi.swift:434); zayıf model

`data_ref=takvim-1` dizesinin tamamını argümana yazabilir, `al()`

`data_ref=` önekini soymuyor → eşleşmez → yukarıdaki sessiz başarı.

- Ref zinciri çok-adımlı ve önizlemede 10 gerçek satır duruyor: model ref taşımak yerine gördüğü 10 satırı doğrudan `icerik`'e yazmaya meylediyor (TakvimAraci.swift:124-126) → 200 kayıtlık takvimden 10 satırlık "tamam görünen" Excel; offload deseni tam varlık nedeniyle deviliyor.
- belge_oku offload dışında: ~375+ token doğrudan bağlama döküyor (BelgeOkuAraci.swift:43-44).

Öneri: Ref çözülemezse ASLA sessiz üretme — açık hata dalı, `al()`

toleransını `data_ref=/kaynakRef=` öneklerini soyacak şekilde genişlet (P0); tek opak ref jetonu + argüman adıyla birebir aynı etiket (P0); export niyetinde önizlemeyi bastır ya da tek-adıma indir; belge_oku'yu da ref kanalına al.

3.8 Hata/retry/iptal semantiği

Mevcut durum: Hata UI'da metin değil bayrakla okunuyor

(ModelServisi.swift:41-53), iptal hata sayılmıyor `akanMetini` koruyor (:891-894), `uretimNo`+defer guard yeni turun bayrağını koruyor (:700,757), yerel yazma retry kilidi doğru (:817-821). Bayrak-tabanlı, sağlam. **En şiddetli zayıflıklar:**

- **Uzak MCP yazmalar retry-güvenliğinin DIŞINDA — çift dış yan etki (KRİTİK):** MCPAraci bilinçle `.okundu` bildiriyor (:227-235), yalnız `.yazildi dünyaDegisti`'yi kuruyor (AracYurutucu.swift:163); uzak "issue aç" başarıyla döndükten sonra model hatası çıkarsa `hataKurtar dünyaDegisti==false` görüp aynı istemi yeniden gönderiyor → çift Jira issue, `tekrarDenenebilir=true` ile üçüncü kopya kapısı bile açılıyor. Asimetri: aynı `.okundu`, onay-kapısını (`oturumKirli`) kirletiyor ama retry-kapısını kirletmiyor.
- `hataKurtar` iptal/`uretimNo` koruması taşıyor: 'dur'+yeni istem senaryosunda eski turun kurtarması `uretiyor=false` iken arka planda yeni bir üretim başlatıp oturumu yeni turla yarıştıyor (ModelServisi.swift:836,849) — görünmez pil yakımı + oturum bozulması.
- Araç→model hata sözleşmesi tümüyle İngilizce sentinel'e dayanıyor (`tool_failed:`, KetumAraci.swift:51; `remote_tool_error:`, AracYurutucu.swift:1092-1099); `hataMi` yine FALSE kalıyor, model ya sentinel'i yankılıyor ya yok sayıp "ekledim" diyor.

- "Yaptım" iddiası ile gerçek araç çağırısı (`dunyaDegisti/izler`) arasında deterministik çapraz kontrol yok (Yonlendirici.swift:39 yalnız talimat).

Öneri: Uzak yan etki için ayrı `disEtki0lusabilir` bayrağı + retry kapısını genişlet (P0); `hataKurtari` tur kimliğine bağla — iptal edilmiş tur kurtarma koşmasını (P0); araç başarısızlığını yapısal `.basarisiz`'dan türet, UI kararı bayraktan gelsin; yan-etki tipini durum enum'undan ayrı modelle.

3.9 Değerlendirme & öz-test kapsamı

Mevcut durum: Deterministik çitler cömertçe assert ediliyor (OtoTestVakalari.swift:96-205, 673-762: hafıza filtreleri, tekilleştirme, enjeksiyon bütçesi, onay kapısı, ağ tekeli); araç doğrulaması modelin anlatısına değil çip izine dayanıyor (AracIzi.swift:29-53); hafıza bloğu "yanlış pozitif=fail, kaçırma=uyarı" ayrımı küçük-model gerçekliğine uygun (Degerlendirme.swift:203-231). **En şiddetli zayıflıklar:**

- **Gerçek-model davranış testleri hiçbir otomatik kapıya bağlı değil (KRİTİK):** Degerlendirme sonucu dosyaya yazıp `print` diyor, eşik/exit yok (:190-192); DilTesti aynı (:126); XCTest hedefi/CI yok. Bu tam olarak "mekanizma kurulur ama üretime bağlanmaz" regresyon deseni — araç seçimi/dil/format bir prompt değişikliğiyle gerilerse hiçbir build kırılmıyor.
- Nondeterminizm bastırılmıyor: `streamResponse(to: soru)` seçeneksiz, GenerationOptions/temperature yok (ModelServisi.swift:867); vaka başına tek koşu (Degerlendirme.swift:119-123) → gerçek gerileme ile örneklem gürültüsü ayrışmıyor.
- Dil çapası otomatik ölçülmüyor, "gözle" bırakılmış (DilTesti.swift:113-115) — küçük modelin imza hatası ölçülmüyor.
- Argüman doğruluğu neredeyse hiç sınanmıyor, yalnız "araç çağrıldı mı": hesap vakaları 1000/135/200 sonucunu doğrulamıyor (Degerlendirme.swift:43-45), takvim-ekle yalnız ikon sayıyor.
- Zayıf iddialar sahte-yeşil üretiyor: zaman-saat `yanitIcermeli: ":"` (her yanıt geçer); ölü `suSaat()` (:544 tanımlı, çağrılmıyor).

Öneri: Model-davranış eval'ini eşikli, exit-code'lu, SAHTE fikstür üzerinde CI kapısına bağla (P0); en düşük sıcaklık + vaka başına N-koşu çoğunluk oranı (P0); dil çapasını NLLanguageRecognizer ile otomatik assert et; argüman doğruluğunu `hamGirdi/hamCikti` üzerinden sına; zayıf iddiaları sıkılaştır.

4. Öncelikli Yol Haritası

P0 — En yüksek kaldıraçlı 5 (küçük-model için kritik)

#	Sorun	Değişiklik	Beklenen küçük-model kazancı
P0-1	Beceri 700-char kör kesmesi somut - <code>tool(args)</code> örneğini + anti-halüsinasyon kurallarını atıyor (BeceriDeposu.swift:78-85; kod.md/web-sayfa.md/belge-olustur.md ölçümleri)	Kesmeyi "çekirdek-önce" modele çevir: örnek + en kritik 1-2 kural gövdenin başına, enjeksiyon yalnız çekirdeği alsın	Beceri artık kendi amacını yerine getirir; kod uydurma / ham HTML / kafadan toplam hataları düşer
P0-2	kaynakRef çözülemezse sessiz başarı: boş/eksik belge "file_created" dönüyor (BelgeOlusturAraci.swift:46-54, 78-87)	Ref dolu ama <code>al()</code> nil ise dosya yazma, açık hata döndür; <code>al()</code> 'ı <code>data_ref=/kaynakRef=</code> örneklerini soyacak şekilde genişlet; ref varken <code>icerik</code> 'ten yazmayı engelle	En tehlikeli hata sınıfı (sessiz veri kaybı) deterministik olarak kapanır
P0-3	Uzak MCP yazmalar retry-korumasının dışında → çift Jira issue/kayıt (MCPAraci.swift:227-235; ModelServisi.swift:846-851)	İkinci yapışkan bayrak <code>disEtkiOlusabilir</code> ; uzak çağrı başarıyla dönünce kur; <code>hataKurtar</code> guard'ını <code>`dunyaDegisti`</code>	\
P0-4	Discriminator/biçim alanları düz <code>String</code> + bulanık <code>.contains</code> ; <code>eylem="add"</code> sessizce okumaya düşüyor (TakvimAraci.swift:19-30; HatirlaticiAraci.swift:185-186; ZamanAraci.swift:21)	eylem/tur/bicim/dil alanlarını @Generable enum'a çevir, <code>.contains</code> dağıtımını switch'e döndür	Geçersiz-operasyon + TR/EN dil-kayması hatası constrained decoding ile deterministik sıfırlanır (3.2/3.3/3.6'yı birden çözer)

P 0 -	Gerçek-model eval'i eşiksiz dosyaya yazıyor, hiçbir şeyi kırmıyor (Degerlendirme.swift:190-192); sampling kontrolü yok (ModelServisi.swift:867)	Eval'i sahte fikstür + en düşük sıcaklık + N-koşu çoğunluk oranı + eşik + non-zero exit ile CI kapısına bağla	Yukarıdaki tüm düzeltmeler regresyona karşı korunur; sessiz davranış gerilemesi build kırar
-------------	--	---	---

P1 — Yüksek değer

Sorun	Değişiklik	Kazanç
Sabit istem ~2100-2400 token, talimat profil-özgü kuralları her profilde taşıyor (Yonlendirici.swift:44-60)	Talimatı çekirdek + profil-eki böl; çekirdeği ~250-300 token'a indir; dil pekiştirmesini tek kanala düşür	Konuşma penceresi açılır, erken özetleme/görünmez unutmaz azalır
Yanlış-profilde tur-içi kurtarma yok (ModelServisi.swift:527-556)	Tur bitince araç izi yok + "yapamadım" kalıbı varsa ikinci en olası profille otomatik yeniden çalıştır	Deterministik seçici yanılınca sessiz yetenek boşluğu kapanır
Tetik alt-dizge eşleşmesi yanlış beceri yüklüyor ('bul' ∈ 'bulutlu', BeceriDeposu.swift:68)	Kelime-sınırlı eşleşme; ≤3-4 karakter tetikler exact-only	Yanlış-kılavuz kaynaklı yanlış davranış düşer
Beceri enjeksiyonu profil araç setinden habersiz (BeceriDeposu.swift:64 vs ModelServisi.swift:320)	Beceriye araclar: etiketi, aktif profile yoksa enjekte etme	"Oturumda olmayan aracı çağır" emri kalkar
Tablo parser çok katı + Excel'de bozuk tablo çöp.xlsx (Tablo.swift:63-65; ExcelMotor.swift:25-28)	Toleranslı parser + nil'de pipe satırlarını .metin koru + Excel'de düzeltme döngüsü	Sessiz kayıp / çöp dosya yerine görünür çıktı
MCP şema genişliği sınırsız + 6-araç sunucu sırasıyla kör dolduruluyor (MCPAraci.swift:380-393; ModelServisi.swift:1037)	Alan/düğüm bütçesi + alaka sıralaması	Token bombası engellenir, kıt yuva alakaya harcanır

hataKurtar iptale kör (ModelServisi.swift:836,849)	Retry öncesi guard uretimNo == benimTur	Görünmez üretim yarışı / oturum bozulması engellenir
Argüman doğruluğu ölçülmüyor (Degerlendirme.swift:43-45)	hamGirdi/hamCikti üzerinden beklenen sonuç/argüman assert et	Doğru araç + yanlış argüman hatası görünür olur
Dil çapası "gözle" (DilTesti.swift:113-115)	NLLanguageRecognizer ile otomatik dil assert	Dil kayması yakalanır

P2 — Sağlamaştırma

Sorun	Değişiklik
Ekli belge kilidi mutlak, kişi/nöbet/kod erişilemez (ModelServisi.swift:530)	Güçlü belge-dışı sinyalde kaçış izni
Beceri bir kez enjekte edilip pencereden kayıyor (ModelServisi.swift:351,415)	"N tur/token mesafesi sonra yeniden enjekte"
Ölü dil alanı zorunlu decode slotu (KodCalistirAraci.swift:42-44)	Kaldır ya da tek-case enum
Araç çıktısındaki İngilizce imperative'ler (MCPAraci.swift:241,248)	Hatayı AracSonucu.durum 'dan yapısal ilet, modele saf veri
Ekli belge varlığı isteme bildirilmiyor (ModelServisi.swift:368-409)	İstemZenginlestir'e [Ekli belge: X – belge_oku çağır] satırı
belge_oku offload dışında (~375 token, BelgeOkuAraci.swift:43-44)	Tam içeriği depoya, modele özet+ref
Bozuk-çıktı ve ref-miss testleri yok	OtoTestVakalari'na sapma matrisi + ref-miss vakaları
Retry sonrası ilk-deneme çipleri kayboluyor (ModelServisi.swift:846)	Birleşik izler
Ad çakışması / MCP per-property açıklama kırpması yok	Dedup+önek; alan açıklamalarına tavan

5. Kesişen Temalar & Riskler

- 1 **Deterministik altyapı var, ama son mil metne/istme bırakılıyor.** Servis→UI hata yolu bayrak-tabanlı (doğru), ama araç→model yolu İngilizce sentinel dizgesine (KetumAraci.swift:51, AracYurutucu.swift:1092-1099); araç seti deterministik seçiliyor ama operasyon argümanı düz **String** (TakvimAraci.swift:30); offload kanalı deterministik ama referansın taşınması istme bırakılmış (Yonlendirici.swift:57). **Kök çözüm ortak: kararın modelden alınabildiği her yerde constrained/yapısal kanala geçmek** — özellikle P0-4 (enum) tek başına 3.2/3.3/3.6'da tekrar eden bulguyu kapatıyor.
- 1 **Token sabit-maliyeti sistematik olarak eksik hesaplanmış.** "~150 token" yorumu gerçekte ~700 (Yonlendirici.swift:5), sonuçta ~2100-2400 token pencere daha başlamadan gidiyor; dil pekiştirmesi üç yerde, araç şemalarında ISO-8601 yönergesi TakvimAraci/HatirlaticiAraci'da tekrarlı (TakvimAraci.swift:21, HatirlaticiAraci.swift:180). Bu, erken özetlemeyi hızlandırıp "görünmez unutma" yaratıyor. Risk: sabit maliyet düzeltilmeden yapılan diğer ayarlar (eşik, korunan tur) yanlış varsayıma oturuyor.
- 1 **Sessiz başarı = en pahalı hata sınıfı ve birden çok boyutta tekrar ediyor.** Ref-miss boş belge "file_created" (3.7), bozuk tablo çöp xlsx "düzenlendi" (3.6), uzak yazma retry çift issue (3.8), hiç araç çağırmadan "ekledim" (3.8) — hepsinde sistem elinde deterministik sinyal (**dunyaDegisti, al()==nil, markdownDan==nil, izler**) olmasına rağmen bunu hata/düzeltmeye çevirmiyor. Küçük model bu sinyalleri kendiliğinden üretemeyeceği için kod-tarafı yakalama şart.
- 1 **"Üretime/koruma altına bağlanmayan mekanizma" regresyon deseni.** Memoide de anılan bu desen en açık 3.9'da: gerçek-model eval'i çalışıyor ama hiçbir kapıyı kırmıyor (Degerlendirme.swift:190-192); ayrıca beceri kesmesi sessiz, ref-miss sessiz, ad-çakışması vaadi tutulmuyor. **P0-5 olmadan diğer dört P0'ın kalıcılığı garanti değil** — bu yüzden eval kapısı listede tutuldu.
- 1 **Katmanlar-arası kopukluk drift üretiyor.** İki paralel anahtar-kelime sistemi (profil izleri vs beceri tetikleri, ModelServisi.swift:596-657 vs .md), profil-bağımsız beceri enjeksiyonu, retry-güvenliği ile görsel-durum ekseninin **.yazildi** tek bayrağına yığılması — hepsi elle senkron gerektiren, tek noktada kayınca küçük-model davranışına doğrudan sızan kopukluklar.

6. Kapanış Hükmü

sirr, ~3B/4096-token bir modelle çalışmak için doğru omurgayı kurmuş: kararı modelden alma, gerçek token ölçümü, deterministik offload ve doğrulama içgüdülerinin hepsi yerinde ve çoğu fiilen işliyor. Mimarının küçük-model uyumu, tasarımın felsefesinde değil, o felsefenin son milinin bırakıldığı yerlerde kırılıyor — sabit istem maliyetinin sessizce şişmesi, **String** discriminator/ref-taşıma/uzak-yazma gibi kararların istme bırakılması ve gerçek-model davranışının hiçbir kapiya bağlı olmaması. Beş PO'nın ortak paydası basittir ve düşük risklidir: **her sessiz-başarı yolunu deterministik bir yakalamaya, her serbest-metin kararı bir constrained kanala bağlamak, ve bu düzeltmeleri eşikli bir eval kapısıyla kilitlemek**. Bu beş hamle, mevcut sağlam altyapıyı bozmadan, küçük modelin en olası sapmalarını görünür ve düzeltilebilir kılar.

Ek A — Boyut Boyut Ayrıntılı Denetim Bulguları

Aşağıdaki tablolar 9 denetçi ajanın ham çıktısıdır (sentezden önce). Bölüm 3'ün kanıt tabanını oluşturur. Zayıflıklar şiddete göre sıralıdır.

A.1 İstem inşası & token bütçesi (ModelServisi/Yonlendirici/BeceriDeposu/HafizaDeposu)

Özet kanaat: Mimarının token-yönetimi altyapısı (progressive disclosure, gerçek tokenCount ölçümü, ayrı özet oturumu, ham araç çıktısını taşımama) küçük model için sınıfının üstünde; ama sabit maliyet iddia edilenin ~4-5 katı (~2100-2400 token) ve talimat profil-özgü kuralları her profilde taşıdığı için 4096 pencerenin yarısı daha konuşma başlamadan tükeniyor.

Mevcut durum. Oturum talimatı **Yonlendirici.talimatlarEN** içinde (Yonlendirici.swift:25-64); dosya başlığı "~150 token" diyor (Yonlendirici.swift:5) ama blok 2833 karakter, yani İngilizce için kabaca ~650-700 token. Talimata ek olarak oturum kurulurken (ModelServisi.swift:423-450) İngilizce dışı her dilde ~872 karakterlik (~200 token) bir "dil çapası" bloğu daha sisteme gömülüyor (ModelServisi.swift:434-445). Buna 6-8 aracın şeması ekleniyor; @Guide açıklamaları oldukça ayrıntılı (ör. TakvimAraci.swift:21 tek argüman için ~230 karakter, HatirlaticiAraci.swift:180 benzer), gündelik profilin 8 aracı (ModelServisi.swift:280-290) kabaca ~1200-1500 token tutuyor. Yani bir Türk kullanıcı için sohbet başlamadan sabit maliyet ~2100-2400 token =

4096'nın ~%50-60'ı. Her tura ise **istemZenginlestir** ile (ModelServisi.swift:368-409) eşleşen tek hafıza notu (≤ 600 krk, HafizaDeposu.swift:22) + tek beceri (≤ 700 krk, BeceriDeposu.swift:26) + non-EN dil hatırlatma satırı (ModelServisi.swift:404) ekleniyor. Bütçe **butceKontrol** ile gerçek **tokenCount** ölçülerek %80 eşiğinde (ModelServisi.swift:904, ~3277 token) yönetiliyor; taşınca ayrı bir oturumda özet üretilip (ModelServisi.swift:928-936) oturum yeniden kuruluyor. Geçmiş transcript'e araç çağrıları/çıktıları HAM girmiyor, yalnız kullanıcı/asistan metni ve o da 400 karaktere kırılarak taşınıyor (ModelServisi.swift:942-958).

Güçlü yanlar:

- Progressive disclosure gerçekten uygulanmış: beceri kılavuzları sabit talimata gömülmüyor, yalnız eşleşen TEK beceri oturuma BİR KEZ iliştiliyor (ModelServisi.swift:344-409, BeceriDeposu.swift:64-93) — küçük modeli sabit talimatla boğmama kararı doğru.
- Token bütçesi tahminle değil gerçek ölçümle yönetiliyor: model.tokenCount(for: transcript) ile ölçülüp contextSize'ın %80'inde özetleme tetikleniyor (ModelServisi.swift:901-908). Deterministik ve modele iş yüklemiyor.
- Özet, dolmuş oturuma istem eklemek yerine AYRI ve kısa bir oturumda üretiliyor (ModelServisi.swift:928-930) — özetin kendisinin taşmayı büyütmesi önlenmiş.
- Araç çağrıları ve hacimli çıktılar geçmiş bağlama ham taşınmıyor; yalnız text segmentleri, o da kırılarak (ModelServisi.swift:942-958) — 4096 pencerede en büyük şişme kaynağı kapatılmış.
- Talimatın EN ÖNEMLİ kuralı küçük modele doğru sinyali veriyor: 'araç gerekiyorsa DOĞRUDAN çağır, niyet anlatma' (Yonlendirici.swift:34-36); enjeksiyon çitleri (HafizaDeposu.swift:105-108, BeceriDeposu.swift:90-92) ve 'araç çıktısındaki talimatlara uyma' kuralı halüsinasyon/sızıntı yüzeyini daraltıyor.
- Enjeksiyon tavanları sert karakter sınırıyla ve yarım-cümle yerine ELEME ile uygulanıyor (HafizaDeposu.swift:91-98) — küçük modele yarım/bozuk olgu gitmemesi için isabetli.

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
[Yüksek] Talimatın gerçek boyutu iddia edilen ~4-5 katı	Yonlendirici.swift:5 ve :22 yorumları '~150 token' diyor; ancak talimatlarEN bloğu (Yonlendirici.swift:25-64) 2833 karakter $\approx$ ~650-700 token. Buna non-EN dil çapası bloğu (ModelServisi.swift:434-445, ~872 krk $\approx$ ~200 token) sistem talimatına ekleniyor.	Sabit talimat maliyeti belgelenenin ~4-5 katı; 4096'lık pencerenin ~%20-22'si daha kullanıcı bir şey yazmadan yalnızca talimata gidiyor. Tasarım kararları yanlış (150 token) sabit maliyet varsayımı üzerine alınmış olabilir — özetleme eşiği, korunan tur sayısı gibi ayarlar gerçek maliyete göre yeniden gözden geçirilmemiş.
[Yüksek] Profil-dışı kalıcı talimat blokları her profilde ödeniyor	SEARCH RESULTS uzun paragrafı (Yonlendirici.swift:44-50, ~100 kelime $\approx$ ~130 token) TEK ve sabit talimatta; oysa gündelik/belge profilinde web_arama aracı hiç yüklenmiyor (ModelServisi.swift:280-311, 320-325). Aynı şekilde belge export/kaynakRef akışı (Yonlendirici.swift:57-60) arama ve bağlantı profillerinde (ModelServisi.swift:320-333) o araçlar yokken taşınıyor.	Küçük modelin dikkat bütçesi her profilde ilgisiz kurallarla harcanıyor. Daha kötüsü: gündelik profilde 'use a search tool if one is listed' ve arama sonuçlarına dair uzun uyarı, modelde OLMAYAN bir arama aracının varlığı beklentisini uyandırabilir → 'aradım ama bulamadım' türü halüsinasyon yüzeyi. Mimari zaten profil başına araç seti tutuyor; talimat bu ayrımı takip etmiyor.
[Yüksek] Sabit maliyet ~%50-60, konuşmaya kalan pencere dar	Talimat ~700 + dil çapası ~200 + 8 araç şeması ~1200-1500 (Araclar/*.swift @Guide açıklamaları uzun, ör. TakvimAraci.swift:21,:23 ve HatirlaticiAraci.swift:180 ISO tarih yönergesi neredeyse birebir tekrarlı) $\approx$ ~2100-2400 token. Özetleme eşiği %80 = ~3277 token (ModelServisi.swift:904).	Sohbet + tur-başı enjeksiyonlar (beceri ~175 + hafıza ~150 token) + kullanıcı sorusu + model çıktısı için pratikte ~900-1200 token kalıyor. Birkaç turda eşik aşılp özetleme tetikleniyor; her yeniden kuruluştaki sabit maliyet (talimat+araç+çapa) YENİDEN ödendiği için model sık sık bağlamını kaybediyor ve kullanıcıya görünmez 'unutma' yaşanıyor.

[Orta] Yanıt dili üç ayrı yerde pekiştiriliyor (token israfı)	1) Talimattaki LANGUAGE paragrafı (Yonlendirici.swift:29-32), 2) oturuma gömülü ~200 token'lık dil çapası bloğu (ModelServisi.swift:434-445), 3) tur-başına '[Reply in X ...]' satırı (ModelServisi.swift:404). İlk ikisi non-EN her oturum kuruluşunda yeniden ödeniyor.	Tek bir kaygıya (yanıt dili) toplam ~350+ token gidiyor ve bu, her oturum yeniden kurulduğunda (dil değişimi, profil geçişi, bütçe taşması) tekrar tekrar ödeniyor. 4096 pencerede bu, konuşma geçmişine ayrılacak alanı doğrudan yiyor; erken özetlemeyi hızlandırıyor.
[Orta] Ekli belgenin varlığı deterministik olarak isteme bildirilmiyor	istemZenginleştir belge adını/varlığını isteme koymuyor (ModelServisi.swift:368-409); ekli belge yalnızca profili .belge'ye KİLİTLİYOR (ModelServisi.swift:530). Model belgeyi ancak eşleşen bir beceri (belge-oku) tetiklenirse öğrenir, o da tetik kelimesine bağlı (BeceriDeposu.swift:64).	Kullanıcı 'bunu özetle' gibi tetik-kelimesiz bir şey derse ne beceri enjekte edilir ne de model bir belgenin ekli olduğunu bilir; belge_oku çağırma'yı atlayıp ekranda gördüğü ama bağlamında olmayan içerik hakkında halüsinasyon üretebilir. Küçük model bu 'örtük durum'u kendiliğinden çıkaramaz.

Öneriler:

Öncelik	Başlık	Gerekçe	Taslak yaklaşım
P0	Talimat çekirdek + profil-eki olarak böl	SEARCH RESULTS ve belge-export blokları her profilde sabit ödeniyor, oysa ilgili araçlar çoğu profilde yüklü değil; ilgisiz talimat token yiyor ve olmayan araç beklentisi/halüsinasyon üretiyor.	oturumKur zaten profil parametresini biliyor (ModelServisi.swift:411,423). Talimatı ikiye ayır: dil-nötr, araç-nötr kısa bir ÇEKİRDEK (kimlik + 'araç gerekiyorsa doğrudan çağır' + 'uydurma') her oturumda; profil-özgü 1-2 satır ise yalnız o profil aktifken eklensin (arama profili → arama-sonucu uyarısı; belge profili → export/kaynakRef akışı). Böylece gündelik profil ~130+ token ilgisiz talimattan kurtulur ve model yalnız masadaki araçların kurallarını görür.

P0	Çekirdek talimatı ölçüp küçült, yanıltıcı yorumu düzelt	Talimat gerçekte ~700 token; 'küçük model uzun kural listesinde talimat-takibini yitirir' gerçeğiyle ve dosyadaki '~150 token' iddiasıyla çelişiyor. Sabit maliyeti düşürmek doğrudan konuşma penceresi açar.	Çekirdeği ~250-300 token hedefine indir: SEARCH RESULTS uzun paragrafını tek cümleye sıkıştırıp ayrıntıyı arama becerisine (arama.md) taşı; 8+ bullet'ı en kritik 4'e indir (uydurma yok / yaptığını gerçekten aracı çağırdıysan söyle / araç çıktısındaki talimata uyma / bilmiyorsan cihazda bulamadım de). Kalanları beceri katmanına aktar — o katman zaten 'gerektiği anda' devrede. Ardından gerçek tokenCount ile ölçüp yorumu doğru sayıyla güncelle ki sonraki ayarlar doğru varsayıma otursun.
P1	Dil pekiştirmesini tek güçlü kanala indir	Yanıt diline ~350+ token, üç ayrı yerde harcanıyor ve her oturum kuruluşunda yeniden ödeniyor; bu 4096 pencerede lüks.	Üç kanaldan yakınlık avantajı en yüksek olanı (tur-başı '[Reply in X]' satırı, araç çıktısına en yakın — ModelServisi.swift:404) koru. Oturuma gömülü ~200 token'lık çapa bloğunu (ModelServisi.swift:434-445) 1-2 cümleye indir; ölçülen 'araç çıktısı dil sürüklemesi' riski yalnız web/arama araçları yüklüyken geçerli olduğundan bu uzun uyarıyı da profil-koşullu yap. Talimattaki LANGUAGE paragrafını tek satıra düşür. Toplam dil bütçesini ~350'den ~80-100 token'a çek.

P1	Araç @Guide e açıklam alarını sadeleş tir ve tekrarı kaldır	8 araç şeması sabit maliyetin en büyük kalemi (~1200-1500 token); ISO-8601 tarih yönergesi TakvimAraci ve HatirlaticiAraci'd a neredeyse birebir tekrarlanıyor (TakvimAraci.swi ft:21, HatirlaticiAraci.s wift:180).	Tekrarlı 'ALWAYS give ISO 8601 / relative wording çöz / zaman aracını çağır' yönergesini @Guide'lardan çıkarıp ilgili beceri kılavuzuna (takvim.md, hatirlatici.md) TEK yerde topla; @Guide'ları tek kısa cümleye indir ('Event time as ISO 8601'). Her aracın name+description+argüman açıklamalarını gerçek tokenCount ile ölçüp en pahalı 2-3 aracı hedefle. Amaç: 8 araç şemasını ~%30-40 küçültmek.
P2	Ekli belge varlığını isteme determi nistik satırla bildir	Tetik-kelimesiz devam isteklerinde model bir belgenin ekli olduğunu bilmiyor; belge_oku'yu atlayıp halüsinasyon üretebiliyor.	istemZenginlestir içinde (ModelServisi.swift:368-409), belgeBaglami.calisilabilirBelge varsa isteme ~15 token'lık deterministik bir satır ekle: '[Ekli belge: rapor.xlsx — içeriğe erişmek için belge_oku çağır]'. Bu, becerinin tetiklenmesine bağlı olmayan, koddan gelen kesin bir sinyal; küçük modelin örtük durumu çıkarmasını beklemekten güvenli. Dil hatırlatması gibi tur-başı bloklarla aynı yerde, aynı 'araç çıktısına yakınlık' avantajıyla akar.

A.2 Niyet profili & araç bütçesi

Özet kanaat: Araç SETİ seçimi deterministik ve bütçe (6-8) fiilen uygulanıyor — mimari doğru omurgaya sahip; ama tam-tavan 8 araçlı profiller, kırılğan ilk-eşleşen anahtar-kelime yönlendirmesi, String operasyon argümanları ve yanlış-profilde tur-içi kurtarma yokluğu küçük modelde sessiz yetenek boşlukları ve halüsinasyon ürettiyor.

Mevcut durum. Araç seti DETERMİNİSTİK seçilir, model seçmez: **niyetProfili** sorunun küçük-harfli metnini 8 dilde anahtar-kelime listeleriyle eşleştirip 4 profilden birini döndürür (ModelServisi.swift:527-556). Profiller ve gerçek araç sayıları: gündelik = 8 araç (Takvim, Hatırlatıcı, Kişi, Arama, Hesap, Zaman, Nöbet, Kod; :280-290), belge = 8 (Oluştur, Oku, Düzenle, Takvim, Hatırlatıcı, Arama,

Hesap, Zaman; :301-311), arama = 3 (web_arama, Hesap, Zaman; :320-325), bağlantı = ilk 6 MCP + Hesap + Zaman = en çok 8 (:329-333, tavan **mcpAracTavani = 6**, :121). Profil içinde 1-of-N aracı MODEL seçer. Öncelik ilk-eşleşen-kazanır sırasıyla: ekli belge kilidi > gündelik izleri > belge izleri > bağlantı > arama > yapışkan mevcut (:530-555). Sonraki turun sinyali modelin metninden değil, çip ikonlarından okunur (:590-593) — bu yön halüsinasyona kapalı. Araç bütçesi 6-8 gerçekten uygulanıyor; ama gündelik/belge tam tavanda (8) duruyor ve kodun kendi yorumu tavanın "ZORLANDIĞINI", cihaz ölçümü kötüyse **zaman**ın düşürüleceğini kabul ediyor (:276-279).

Güçlü yanlar:

- Araç SETİ deterministik seçilir; küçük model hangi araçların masada olduğunu seçmez, yalnızca dar bir set içinden tekini seçer — yanlış-set hata yüzeyi tamamen kapatılmış (ModelServisi.swift:527-556).
- Araç bütçesi (6-8) fiilen uygulanıyor: hiçbir profil 8'i aşmıyor, MCP tarafında tavan çeviriden ÖNCE uygulanıp 200 araçlık sunucuda bile 6'ya iniyor (:1037, :121).
- Cihaz-dışı profiller (arama, bağlantı) kişisel veri araçlarını YAPISAL olarak dışlıyor: araç oturumda yoksa model yanlışlıkla da olsa veriyi argümana yazamaz (:313-333).
- Sonraki-tur yönlendirme sinyali modelin ürettiği metinden değil, aracın düşürdüğü çip ikonundan okunuyor — takip sorusu ("peki yarın?") için halüsinasyona kapalı, deterministik bir kanal (:587-593).
- Gereksiz oturum yeniden kurmayı önlemek için mevcut profil isteği karşılıyorsa değişmiyor; profil geçişi = transcript'i koruyarak tek seferde (:524-526, :780).
- Araç argümanları @Generable/@Guide ile tipli; model serbest metin üretip parse ettirmiyor (KetumAraci.swift:5-7).

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
----------	-------	--------------------

[Yüksek] Çapraz-sinyal çakışması yanlış profile kilitliyor: ilk-eşleşen-kazanır yerel dışa aktarmayı öldürüyor	ModelServisi.swift:539 if Self.gundelikIzleri.contains(where: s.contains) { return .gundelik } belge izlerinden ÖNCE bakıyor; gundelikIzleri "notlarım"ı içeriyor (:597), gündelik profilde ise belge_olustur YOK (:289).	"notlarımı excel dosyası yap" gibi tek cümlede iki sinyal var; metin önce "notlarım"a takılıp gündelik profile düşüyor, o profilde dışa-aktarma aracı bulunmadığı için 3B model ya işi yaptığını halüsine ediyor ya da "yapamadım" diyor — kullanıcının açık dosya isteği sessizce kayboluyor. Öncelik yorumu (:532-538) yalnız MCP iki-aşamalı akışını düşünüp yerel belge çakışmasını gözden kaçırmış.
[Yüksek] Yanlış-profil durumunda tur-içi kurtarma yok — sessiz yetenek boşluğu	niyetProfili tek atışta profili seçiyor (:527-556); seçtikten sonra oturuma yalnız o profilin araçları giriyor (:423) ve model "başka araç lazım" diye sinyal veremiyor, geri-yönlendirme yolu yok.	Deterministik seçici yanılırsa (ki 8 dilli anahtar-kelime eşleşmesi kırılğan) ihtiyaç duyulan araç o turda hiç oturumda değil. Küçük model eksik aracı çağıramayacağı için ya niyeti anlatmakla yetiniyor ya da başarıyı uyduruyor — talimatın en çok korktuğu halüsinasyon tam burada, çünkü hiçbir deterministik güvence devrede değil.
[Orta] gündelik ve belge profilleri tam 8 araçla tavanda — 1-of-8 ayrımı küçük modelin en zayıf noktası	gundelikAraclar() 8 araç döndürüyor (:289), belgeAraclar() 8 (:310); kodun kendi yorumu "8 araçla tavan ZORLANIYOR ... cihaz ölçümü kötü çıkarsa zaman düşürülür" diyor (:276-279).	3B/düşük-bit model için doğru aracı seçme oranı araç sayısı ile hızla düşer; 8 araç pratik üst sınırın ucunda. Nöbet ve Kod gibi seyrek kullanılan araçlar her turda sette durup Takvim/Hatırlatıcı gibi sık araçlarla karışma ihtimalini artırıyor — özellikle argüman şemaları benzer araçlarda (takvim vs hatırlatıcı: ikisi de eylem+başlık+zaman).

[Orta] Operasyon argümanları @Generable enum değil serbest String; dağıtım kırılğan . contains ile	TakvimAraci.swift:20 var eylem: String ve :29 arguments.eylem.lowercased().contains("ekle"); HatirlaticiAraci.swift:177 var eylem: String, dağıtım `contains("oku")\`	\
[Orta] Ekli belge kilidi mutlak — kişi/ nöbet/ kod belge varken tamamen erişilemez	ModelServisi.swift:530 if belgeBaglami.calisilabilirBelge != nil { return .belge } tüm anahtar-kelime kontrollerinden ÖNCE koşulsuz dönüyor; belgeAraclar() Kişi/Nöbet/Kod içermiyor (:301-311, yorum :292-300 KisiAraci'nin bilerek çıkarıldığını söylüyor).	Bir belge ekliyken kullanıcı "bunu her sabah nöbetle gönder" ya da kod/kişi isteği yaparsa profil belgeye kilitli kaldığı için o araçlar oturumda hiç yok; model isteği karşılayamıyor ama sebebini de bilmiyor, muhtemelen yaptığını uyduruyor. Kilit yapışkanlığı küçük modelde çıkışsız bir çıkmaz yaratıyor.
[Düşük] Bağlantı profili en zor ayrım vakasını küçük modele yıkıyor: 6 tanıdık-olmayan uzak araç + 2 genel araç	baglantiAraclar() mcpAraclari.prefix(6) + [hesap, zaman] = 8 araç (:329-332); MCP şemaları sunucudan çalışma anında geliyor (:1021-1055), özet açıklamayı kısaltıyor ama parametre SAYISINI değil.	Server-tanımlı, isimleri/şemaları modelin eğitiminde olmayan 6 uzak aracın arasına ayrıca hesap+zaman karışınca 3B model için doğru-arac seçimi en düşük orana iniyor; üstelik uzak araçlar sık sık hesap/zaman ile aynı turda gerekmiyor, yani iki genel araç boş yere ayrım yükünü artırıyor.

Öneriler:

Ön celi k	Başlık	Gerekçe	Taslak yaklaşım
P0	Tur-içi yanlış-profil kurtarma kapısı ekle	En büyük küçük-model riski, deterministik seçicinin yanlış gereken aracın oturumda hiç olmaması ve modelin bunu halüsinasyonla örtmesi (zayıflık 2). Bugün hiçbir güvence yok.	yanıtSonucu içinde tur bittikten sonra deterministik bir kontrol: eğer profil gündelik-dışı bir profile yönlendirildiyse VE turda hiç araç izi düşmediyse (yurutucu.izler boş) VE yanıt bir 'yapamadım/niyet' kalıbına benziyorsa, isteği bir kez daha gündelik (veya sinyal skorlamasına göre en olası ikinci) profile, taze oturumda otomatik yeniden çalıştır. Bu, mevcut hataKurtar altyapısına paralel, tamamen kod-tarafı bir emniyet ağı; modelden hiçbir sinyal istemez.
P1	Operasyon argümanlarını @Generable enum'a çevir	String eylem/tur, çerçevenin kısıtlı-çözümleme garantisini boşa harcıyor; .contains heuristiği model beklenmedik değer üretince yanlış yola sapıyor (zayıflık 4).	Takvim.eylem, Hatirlatici.eylem, Zaman.tur ve Kod.dil için String yerine @Generable enum (ör. enum Eylem { case oku, ekle }) tanımla. Çerçeve üretimi yalnızca geçerli case'lere zorlar; .contains dağıtımı kaldırılıp doğrudan switch'e döner. Kod yazma değil; şema tipini String'den enum'a taşıma tasarım kararı — küçük modelde geçersiz-operasyon hatasını deterministik olarak sıfırlar.
P1	Profil seçimini ilk-eşleşen-kazanır'dan sinyal-skorlamasına taşı	Çapraz-sinyal çakışması yerel dışa-aktarmayı öldürüyor: "notlarımı excele dök" gündeliğe düşüp belge_olustur'u kaybediyor (zayıflık 1).	niyetProfili'nde her profil için eşleşme gücü hesapla ve en güçlü/en-özgöl sinyali seç. Biçim adları (excel/pdf/docx) gibi güçlü belge sinyalleri, genel "notlarım" gibi zayıf gündelik sinyallerini yenmeli. MCP iki-aşamalı akışın gerektirdiği gündelik-önceliği yalnızca gerçek çakışmada (bağlantı sinyali + gündelik sinyali birlikte) koru; yerel belge çakışmasında biçim sinyali kazansın.

P1	gündeli k/belge profilleri ni 8'den 5-6 araca indir; Nöbet+ Kod'u tetiklem eli ayrı profile al	8 araç küçük modelin doğru- araç oranının düştüğü tavan; kodun kendi yorumu da bunu kabul ediyor (kod-spec §9 açık soru 2). Nöbet ve Kod seyrek ama her turda ayırım yükü ekliyor (zayıflık 3).	Nöbet ve Kod'u kendi anahtar-kelime izleriyle tetiklenen ayrı profillere (veya tek 'otomasyon' profiline) taşı; gündelik profili Takvim/Hatırlatıcı/Kişi/Arama/Hesap/ Zaman'a (6) indir. Böylece varsayılan turda model daha az, birbirine daha az benzeyen araç arasından seçer. niyetProfili'ne 'nobet'/'kod' izleri eklemek yeterli; mevcut deterministik seçici altyapısı aynen kullanılır.
P2	Ekli belge kilidini yumuşat: güçlü belge-dışı sinyalde kaçış izni ver	Mutlak kilit, belge ekliyken kişi/nöbet/kod'u tamamen erişilemez yapıp küçük modeli çıkışsız bırakıyor (zayıflık 5).	niyetProfili:530'daki koşulsuz return .belge 'yi koşullu yap: yalnız güçlü belge-dışı sinyal (nobet/kod/kişi izi) yoksa belgeye kilitle. Güçlü belge-dışı sinyal varsa ilgili profile geçmeye izin ver — 'onu tablo yap' gibi biçimsiz devam istekleri hâlâ belgede kalır çünkü onlarda karşı-sinyal yoktur. Kilit yapışkanlığının çıkmaz yaratmasını engeller.

A.3 Araç şemaları & açıklamaları (@Generable/@Guide)

Özet kanaat: Constrained decoding ve tek-string araçlar sağlam bir temel; ama yerli araçların discriminator/biçim alanlarının gerçek enum yerine düz String olması ve TR/EN dil karışımı, 3B modelin en sık tökezleyeceği somut halüsinasyon yüzeylerini açık bırakıyor.

Mevcut durum. Tüm yerli araçlar FoundationModels **Tool** + **@Generable/ @Guide** ile tip-güvenli tanımlı; model serbest metin üretip parse ettirmiyor, constrained decoding var (KetumAraci.swift:6-7). Argümanların çoğu TEK zorunlu string: Hesap **ifade** (HesapAraci.swift:20-23), Kişi **isim** (KisiAraci.swift:12-15), Arama **anahtar** (AramaAraci.swift:18-21), WebArama **sorgu** (WebAramaAraci.swift:30-33) — küçük model için ideal sadelik. Açıklamalar İngilizce ve çoğunda somut örnek var ("e.g. 'istanbul weather tomorrow'"), bu bilinçli bir seçim (Yonlendirici.swift:8-10 rapor §5.4: İngilizce talimat küçük modelde daha iyi). Ancak ayırım (discriminator) ve biçim alanları gerçek enum DEĞİL, düz **String** + "use these exact values" nasihatıyla veriliyor ve kod

tarafında `.lowercased().contains("ekle")` gibi bulanık eşleştirmeye çözülüyor (ZamanAraci.swift:21, TakvimAraci.swift:19-30, HatirlaticiAraci.swift:176,185-186, BelgeOlusturAraci.swift:24). Takvim/Hatırlatıcı iki eylemi (oku/yaz) tek araca `eylem` string'iyle katlıyor; koşullu-zorunlu alanlar yalnız düzyazıda anlatılıyor. Profil başına aktif araç sayısı 3-8 tutuluyor (ModelServisi.swift:289,310,324,332). MCP tarafında şema çalışma anında çevriliyor: derinlik sınırı 4, düzleşmeyen araç atlanıyor, sayı tavanı 6, ve araç açıklaması 160 karaktere özetleniyor (MCPAraci.swift:365-424, BaglantiServisi.swift:310-327).

Güçlü yanlar:

- Constrained decoding: model şemaya aykırı argüman üretemez; hiçbir yerde serbest-metin ayrıştırma yok (KetumAraci.swift:6-7). MCP'de bile Arguments=GeneratedContent olarak şemaya kilitli (MCPAraci.swift:122-124).
- Araçların çoğu tek zorunlu string alan taşıyor (Hesap/Kişi/Arama/WebArama) — 3B model için en güvenli şema biçimi; dolduracak tek slot, halüsinasyon yüzeyi minimum.
- @Guide açıklamaları kısa ve neredeyse hepsinde somut örnek içeriyor (ZamanAraci.swift:21, HesapAraci.swift:22, WebAramaAraci.swift:31), modelin doğru biçimi taklit etmesini kolaylaştırıyor.
- Açıklamalar İngilizce — küçük on-device modelin talimat-takibi İngilizcede daha güçlü (Yonlendirici.swift:8-10) ve eval ile seçilmiş.
- MCP şeması küçük modele karşı savunmalı: derinlik sınırı 4 (MCPAraci.swift:368), düzleşmeyen araç atlanıyor, sayı tavanı 6 (ModelServisi.swift:121), açıklama 160 karaktere kırılıyor (BaglantiServisi.swift:314-326) — pencere tek araçla dolmuyor.
- MCP enum'ları gerçek constrained seçenek listesine çevriliyor (MCPAraci.swift:406-412 anyOf): model listenin dışına çıkamıyor.
- Profil kilidi sayesinde model bir anda en fazla 8 araç görüyor; küçük model için araç kalabalığı bilinçli sınırlanmış (ModelServisi.swift:289,310,324).

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
----------	-------	--------------------

[Yüksek] Yerli araç enum'ları gerçek @Generabl e enum değil, düz String + bulanık eşleştirme	TakvimAraci.swift:19-20 @Guide(..."oku"/"ekle"...) var eylem: String ve satır 30 let ekleMi = arguments.eylem.lowercased().contains("ekle"); HatirlaticiAraci.swift:185-186 `okuMu = eylem.contains("oku") \	\
[Yüksek] belge_olus tur şeması: string içine gömülü markdown tablo + düzyazıyla anlatılan karşılıklı dışlama	BelgeOlusturAraci.swift:30 icerik için \	Header1 \
[Orta] TR/ EN dil karışımı: alan adları Türkçe, değerler Türkçe, açıklamalar İngilizce; tek enum içinde diller karışık	BelgeOlusturAraci.swift:24 bicim değerleri 'excel','pdf','word','markdown','me tin','html' — 'metin' Türkçe, gerisi İngilizce, aynı listede karışık. ZamanAraci.swift:21 tur='saat'/'tarih'/'gun'/'hepsi' hepsi Türkçe ama açıklama İngilizce; alan adları tur/eylem/bicim/sorgu Türkçe.	Model (ör. İngilizce sohbette) İngilizce @Guide okuyup Türkçe-adlı alana Türkçe literal değer basmak zorunda. Bu diller-arası eşleme tam da 3B zayıf modelin şaşırdığı yer; 'text' ya da 'plain' üretip 'metin'i ıskalayabilir. Tek enum içinde dilin tutarsızlığı geçersiz değer olasılığını artırır.

[Orta] kod_calisti r'da dil alanı ölü: motor yok sayıyor ama zorunlu decode slotu	KodCalistirAraci.swift:42-44 @Guide(description: "js") var dil: String; satır 70 yorumu 'dil ne derse desin JSC çalışır'; §5.1 'python istense de JS ile çözülür'.	İşe yaramayan zorunlu bir alan; @Guide'ı harfiyen 'js' (anlamsız tek kelime). Küçük model her zorunlu alandaki takılabilir/halüsine edebilir — 'python', 'swift' doldurur, sonuca hiç etkisi yok ama decode bütçesi ve hata yüzeyi harcanır. Alan tamamen çıkarılsa şema iki alandan tek alana (kod) iner.
[Orta] Aşırı yüklü çok- eylemli araçlar: koşullu- zorunlu alanlar yalnız düzyazıda	TakvimAraci.swift:18-27: eylem + baslangic('Required for ekle') + bitis + baslik tek şemada; koşullu zorunluluk sadece @Guide metninde. HatirlaticiAraci.swift:175-182 aynı desen.	Küçük model önce doğru eylem string'ini seçmeli, sonra o eyleme göre hangi alanların zorunlu olduğunu düzyazıdan çıkarmalı — iki katmanlı koşullu akıl yürütme. 'ekle' derken baslangic'ı boş bırakması ya da 'oku' derken baslik uydurması olası; düz, tek- amaçlı araçlar bu yükü ortadan kaldırırdı.
[Düşük] MCP per- property şema açıklamalar ı kırpılmıyor (yalnız araç- düzeyi özet 160'a iniyor)	MCPAraci.swift:389 description: alt["description"] as? String — sunucunun her alan açıklaması ham geçiyor; kırpma yalnız araç açıklamasında (BaglantiServisi.swift:314).	Çok alanlı, uzun per- property açıklamalı bir uzak sunucu, 4096 pencereye tek araçla ciddi yük bindirebilir; hesap+zaman ile birlikte 6 uzak araç girince bağlam şişer ve model araç seçiminde kararsızlaşır.

Öneriler:

Ön celi k	Başlık	Gerekçe	Taslak yaklaşım
-----------------	--------	---------	-----------------

P0	Yerli discrimina tor/biçim alanlarını gerçek @Generable enum'a çevir	En büyük şema zayıflığı: eylem/tur/bicim düz String olduğu için şema değeri kısıtlamıyor ve kod bulanık .contains() ile yanlış dala düşebiliyor (TakvimAraci.swift:30 eylem="add" → sessiz okuma). @Generable Swift enum kullanıldığında constrained decoding modeli yalnız geçerli case token'larına zorlar; bütün bulanık-eşleştirme ve dil-kayması sınıfı deterministik olarak kapanır.	eylem, tur, bicim, kod_calistir örneğinde ilgili tüm ayırım/biçim alanlarını String yerine @Generable enum (Generable'a uyan Swift enum: oku/ekle, saat/tarih/gun/hepsi, excel/pdf/word/... case'leri) olarak tanımla. Böylece model şema düzeyinde geçerli değeri üretmeye kilitlenir, koddaki .lowercased().contains bulanık eşleştirmeleri kaldırılabilir. Zaten MCP tarafında enum'lar bu şekilde (anyOf ile) constrained; yerli araçlar aynı garantiye kavuşur.
P1	kod_calistir'daki ölü dil alanını kaldır	Motor değeri tamamen yok sayıyor (§5.1) ama zorunlu bir decode slotu ve anlamsız @Guide='js' modelin takılıp halüsine edebileceği bir alan. Küçük modelde her gereksiz zorunlu alan risk.	Arguments'ı tek alana (kod) indir; dil ayırımı gerçekten gerekirse tek-case bir @Generable enum yap ki model üretmesin. Şema iki alandan tek alana inince araç çağrısı belirgin sadeleşir.
P1	Enum değerleri ile açıklama/ alan dilini tekleştir	Alan adları Türkçe, değerler Türkçe, açıklamalar İngilizce ve tek enum içinde diller karışık (BelgeOlusturAraci.swift:24 'metin' vs 'markdown'). Diller-arası eşleme 3B modelin en zayıf olduğu yer; geçersiz değer olasılığını artırıyor.	İki tutarlı yoldan birini seç: (a) @Generable enum'a geçilince literal değer artık modelden geçmez, sorun kökten çözülür — tercih edilen yol; ya da (b) enum kalacaksa tüm case'leri tek dile çek (hepsi İngilizce: text/markdown/html...) ki model açıklama diliyle değer dili arasında çeviri yapmak zorunda kalmasin.

P1	belge_olustur'da string-içi tablo kalıbını yapısal kanala taşı	JSON string alanının içine elle hizalı markdown tablo kurmak küçük model için kırılğan; icerik/kaynakRef karşılıklı dışlaması yalnız düzyazıda ve model ikisini birden ya da hiçbirini doldurabilir.	Toplu/tablo veriyi mümkün olan her yerde kaynakRef (VeriDeposu) üzerinden zorunlu kıl; serbest metin için icerik'i düz prose ile sınırla. Tablo üretimini modelin string içinde boru-tire dizmesine bırakmak yerine, tablo verisini her zaman bir kaynak aracın döndürdüğü ref'ten çekecek biçimde tasarla; böylece modelin ürettiği tek şey format seçimi ve dosya adı olur.
P2	Çok-eylemli araçları böl ya da kod-tarafı eşleştirme yi dayanıklılaştır	Takvim/Hatırlatıcı tek araçta oku+yaz katlıyor; koşullu-zorunlu alanlar düzyazıda kalıyor ve küçük model iki-katmanlı koşulu doğru kuramayabiliyor. Ayrıca yanlış eylem değeri sessiz yanlış dala düşüyor.	İki seçenek: (a) araçları ayır (takvim_oku/takvim_ekle) — her biri düz, koşulsuz zorunlu alan setine sahip olur, ama profil araç sayısını artırır, bütçeyle tart; ya da (b) tek araç kalacaksa enum'a geç ve tanınmayan/eksik eylemde SESSİZ varsayılanla düşmek yerine modele açık 'geçersiz eylem, şu değerlerden birini ver' hatası döndür (bugün TakvimAraci.swift:30 herhangi bir tanınmayan değeri okumaya çeviriyor).
P2	MCP per-property açıklamalarına uzunluk tavanı koy	Araç-düzeyi açıklama 160'a kırılıyor ama alan açıklamaları ham geçiyor (MCPAraci.swift:389); uzun açıklamalı çok-alanlı bir sunucu 4096 pencereyi tek araçla şişirebilir.	Şema çevirisinde her Property.description'ı da (araç özetiyle aynı mantıkla) tek satıra ve makul bir tavana kırp; böylece 6 uzak araç girince toplam şema yükü öngörülebilir kalır ve model araç seçiminde kararlı olur.

A.4 MCP köprüsü & dinamik araç yükü

Özet kanaat: Araç sayısı tavanı, tek-bağlantı seçimi, özetleme ve derinlik filtresi küçük model için sağlam kurulmuş; asıl açık, şema GENİŞLİĞİNİN sınırsız olması ve 6-araç bütçesinin alaka yerine sunucu sırasıyla kör

doldurulması.

Mevcut durum. Uzak MCP sunucusundan gelen araçlar dört katmanlı bir tavanla eleniyor: istemcide `enFazlaArac = 200` ve `enFazlaSayfa = 20` sayfalama sınırı (MCPIstemcisi.swift:105-109,215), profilde `mcpAracTavani = 6` (ModelServisi.swift:121), aynı anda YALNIZCA tek bağlantı seçiliyor (ModelServisi.swift:145-149) ve tavan şema çevirisinden ÖNCE uygulanıyor (`.prefix(tavan)`, ModelServisi.swift:1037). Yani 30 araç dönen bir sunucudan modele en fazla 6 uzak araç + Hesap + Zaman = 8 araç giriyor; küçük model 30 araca hiç boğulmuyor. Tanım özetleme GERÇEKTEN uygulanıyor: ekleme anında her açıklama on-device modele ~20 kelime/160 karaktere indirtilip `aracOzetlerinde` önbellekleniyor (BaglantiServisi.swift:309-327), oturuma yalnız bu özet giriyor (MCPAraci.swift:158). Şema derinliği `derinlikSiniri = 4` ile sınırlı, `anyOf/oneOf/allOf/$ref` reddediliyor ve çevrilemeyen araç sessizce yutulmayıp "desteklenmiyor" diye listeleniyor (MCPAraci.swift:327,368; BaglantiServisi.swift:345-366). Kirli-oturum onay kapısı kodda tutuluyor, model bayrağı göremiyor (AracYurutucu.swift:50-53,87-112) ve bağlantı profili kişisel veri araçlarını yapısal olarak dışarıda bırakıyor (ModelServisi.swift:327-333).

Güçlü yanlar:

- Araç sayısı tavanı çok katmanlı ve çeviriden ÖNCE uygulanıyor: 30 (hatta 200) araçlık sunucu modele asla 6'dan fazla uzak araç vermiyor (ModelServisi.swift:1037, MCPIstemcisi.swift:109, ModelServisi.swift:121); tek bağlantı seçildiği için birden çok sunucu da bütçeyi şişirmiyor (ModelServisi.swift:145-149).
- Tanım özetleme fiilen var ve önbellekli: ham 100-500 token'lık açıklama modele hiç girmiyor, ~20 kelimeye indirgenmiş özet giriyor (BaglantiServisi.swift:314-327, MCPAraci.swift:157-158) — 4096 pencerede tek aracın tanımı pencereyi doldurmuyor.
- Şema derinliği deterministik olarak sınırlanıyor (kalan:4) ve birleşim/özyineleme tipleri (`anyOf/oneOf/allOf/$ref`) reddedilip araç atlanıyor; küçük modelin dolduramayacağı derin ağaç hiç oturuma girmiyor (BaglantiServisi.swift:350-366, MCPAraci.swift:365-424).
- Constrained decoding sayesinde model şemaya aykırı argüman üretmiyor; derleme zamanı tip olmadan çalışma anı şema kuruluyor (MCPAraci.swift:9,119-124).
- Onay kapısı modelde değil kodda; ret önbelleği (`reddedilenKaynaklar`)

modelin ısrar döngüsüne girmesini yapısal olarak engelliyor (AracYurutucu.swift:58,88-92,108). Bağlantı profili kişisel veri araçlarını hiç içermeyerek tek-adımda sızmayı imkânsız kılıyor (ModelServisi.swift:327-333).

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
[Yüksek] Şema GENİŞLİĞİ (alan/dal sayısı) hiç sınırlanmıyor, yalnız derinlik	MCPAraci.swift:380-393 — object dalında <code>alanlar.keys.sorted().map</code> tüm alanları koşulsuz şemaya koyar; tek bir üst sınır yalnız derinlik (derinlikSiniri=4, satır 327). BaglantiServisi.swift:350-360 derinlikUygun de yalnız derinliği sayar, alan sayısını değil.	Derinlik 1 ama 40 alanlı tek bir object şeması sınırı geçer ve olduğu gibi modele girer. 3B/4096 model 40 alanlı tek çağırışı doğru dolduramaz: alanları uydurur, zorunlu olanları atlar ya da JSON'u yarıda keser. Tavan '6 araç' olsa da her aracın şeması genişlik yönünden token bombası olabilir.
[Yüksek] Bütçeye giren 6 araç alakaya göre değil, sunucunun tools/list sırasına göre seçiliyor	ModelServisi.swift:1037 — <code>tanımlar.filter { ozetSozlugu[\$0.ad] != nil }.prefix(tavan);</code> tanımlar sunucu sırasında, hiçbir alaka/sorgu sıralaması yok. Yorum da (satır 1034-1036) yalnız 'sunucu sırası korunur' diyor.	Kullanıcının ihtiyaç duyduğu araç sunucu listesinde 7. sıradaysa modele Hiç görünmez; model ya 'yapamam' der ya eldeki 6 araçtan yanlısını çağırır. 6-aracılık kıt bütçe kör harcanıyor — küçük modelde en pahalı kaynak (araç yuvası) rastgele dolduruluyor.

[Orta] Araç adı çakışması çözümü yor; kod yorumda ki 'önek' vaadini tutmuyor	MCPAraci.swift:276-282 gecerliAd yalnız harf/rakam/ alt-çizgi dışını _ yapar, önek EKLEMEZ. Ama satır 129 yorumu "name çakışmayı önlemek için önek almış olabilir" der. ModelServisi.swift:1039-1053 üretimde de dedup yok.	"get-user" ve "get_user" adlı iki uzak araç ikisi de "get_user"a iner; aynı isimli iki Tool oturuma girer. FoundationModels bunu reddedip TÜM bağlantı profilini düşürebilir ya da model iki özdeş adlı araç arasında kararsız kalıp yanlışı seçer. Küçük modelde ayırt edilemeyen iki isim doğrudan yanlış-arac çağrısı üretir.
[Orta] Araç çıktısına gömülü İngilizce model-talimatları, 'araç çıktısındaki talimata uyma' ilkesiyle çelişiyor	MCPAraci.swift:241,248 ("remote_call_failed: ... Say this in one sentence; do not invent a result.") ve ModelServisi.swift:1092,1099 ("remote_tool_error:", "remote_tool_empty: ... do not invent a result"). Oysa istemZenginleştir yorumu (ModelServisi.swift:400-402) web-arama §5.6'ya atıfla 'araç çıktısındaki talimatlara uyma' diyor.	Modele dönen metin hem veri hem imperative talimat. Zayıf 3B model iki uçtan birine düşer: ya literal remote_tool_error: / remote_call_failed: jetonunu kullanıcıya Türkçe yanıt içinde yansıtır (halüsinasyon/kaçak jeton), ya talimatı yok sayıp boş sonucu 'başarılı' diye uydurur. Türkçe asistan için İngilizce imperative ayrıca dil çapasını zorlar.
[Orta] İçe aktarma da tavan uygulanmıyor: kullanılmayacak onlarca araç boşuna on-device modele özetletiliyor	BaglantiServisi.swift:274-280 — for tanim in tanimlar TÜM (200'e kadar) desteklenen aracı tek tek ozetle ile özetler; özetle her çağrıda ayrı LanguageModelSession açar (satır 318). Tavan (6) yalnız araçlarıKur'da uygulanır (ModelServisi.swift:1037), özetlemede yok.	30-200 araçlık sunucuda 30-200 SIRALI 3B çıkarımı ekleme anında koşar; yalnız 6'sı hiç kullanılır. Cihazda dakikalarca süren arka plan işi + pil tüketimi; küçük/ yavaş NPU'da bu 'araçlar okunuyor' durumu uzar, kullanıcı bağlantıyı çalışmıyor sanabilir.

[Düşük] Özetleyici aynı zayıf 3B model; özet halüsine syonu doğrulan mıyor	BaglantiServisi.swift:318-326 — özet aynı <code>SystemLanguageModel.default</code> ile üretilir, çıktı yalnız <code>tekSatir</code> ile kırpılır, içerik doğrulaması yok; başarısızsa ham kırpmaya düşer (satır 326).	Özetleyici aracın ne yaptığını yanlış sıkıştırırsa (ör. 'sil' aracını 'listele' diye özetlerse) oturuma yanlış tanım girer ve model aracı yanlış amaçla çağırır. Özet, aracın modele görünen TEK tanımı olduğu için hata tek kaynakta kalıcılaşıyor.
[Düşük] Async araç kurulum u yarışı: tanımlar yüklenm eden gelen ilk istek bağlantı profilini seçemez	ModelServisi.swift:140 ("araçlar hazır olana kadar <code>mcpAraclari</code> boştur"), 563 (<code>baglantiKullanilabilir: Bool { ! mcpAraclari.isEmpty }</code>), 543 (profil yalnız kullanılabilirse seçilir).	Kurulum bitmeden kullanıcı 'sunucuda issue aç' derse profil gündelikte kalır, uzak araç modele görünmez; model bağlantısız kalır ama işi yaptığını uydurabilir. Yarış nadir ama sessiz: kullanıcı çipsiz bir 'oldu' yanıtı görür.

Öneriler:

Ön celi k	Başlık	Gerekçe	Taslak yaklaşım
-----------	--------	---------	-----------------

P1	Şema genişliği için alan/düğüm bütçesi ekle	Derinlik sınırı tek başına yeterli değil; küçük modeli asıl boğan şey tek turda doldurması gereken alan SAYISI ve toplam şema jeton kütlesi. 40 alanlı düz bir object derinlik-4 filtresini geçip token bombası olarak oturuma giriyor (MCPAraci.swift:380-393).	MCPSemaCevirici.dugum'a derinliğin yanına iki deterministik sınır daha koy: (1) tek object başına maksimum alan sayısı (ör. 8-10), (2) tüm ağaç boyunca toplam düğüm/leaf sayısı tavanı. Sınır aşılsa aracı — derin şemada olduğu gibi — atlayıp 'şeması fazla geniş' nedeniyle 'desteklenmiyor' listesine düşür. semaDesteklenirMi'ye (BaglantiServisi.swift:345) aynı genişlik kontrolünü ekle ki desteklenmeyen araç özetlemeye de girmesin. Böylece modele giden her uzak araç hem sığ hem dar kalır.
P1	6-araç bütçesini sunucu sırası yerine alakaya göre doldur	Kıt 6-araç yuvası şu an sunucunun tools/list sırasına göre kör dolduruluyor (ModelServisi.swift:1037); ihtiyaç duyulan araç listede geç sıradaysa modele hiç görünmüyor. Küçük modelde araç yuvası en pahalı kaynak, rastgele harcanmamalı.	araclariKur'a giren adayları prefix(tavan) ile kesmeden önce hafif, deterministik bir alaka sıralaması uygula: kullanıcının o turdaki sorusundaki sözcükler ile araç adı/özetinin kesişimi + sonKullanım benzeri 'bu araç yakında çağrıldı mı' sinyali. Model çağrısı gerektirmeyen saf metin eşleşmesi yeter. En alakalı 6'yı seç; kalanları oturum dışında bırak ama bağlantı detayında görünür tut. Alaka sinyali yoksa mevcut sunucu-sırası davranışına düş (geriye dönük güvenli).
P2	Araç adı çakışmasını dedup + gerçek önekle çöz	gecerliAd sanitizasyonu iki farklı uzak adı aynı ada indirgeyebiliyor ve kod, satır 129'daki 'önek almış olabilir' vaadini tutmuyor (MCPAraci.swift:276-282). Aynı isimli iki Tool ya profili düşürür ya modeli yanlış araca yönlendirir.	araclariKur döngüsünde (ModelServisi.swift:1040-1053) üretilen nameleri bir Set ile izle; çakışma olursa deterministik bir sonek (ör. _2) ekle ya da bağlantı-kısa-önekiyle adlandır. uzakAd gerçek çağrı için zaten ayrı tutuluyor, dolayısıyla görünen ad değişse de ağ çağrısı bozulmaz. Yorumu da koda hizala: ya öneki gerçekten uygula ya vaadi yorimdan kaldır.

P2	Araç çıktısındaki İngilizce model-talimatların yapısal durum a taşı	modeleDonen metnine gömülü 'remote_call_failed: ... Say this in one sentence' gibi imperative'ler (MCPAraci.swift:241,248; ModelServisi.swift:1092,1099), projenin kendi 'araç çıktısındaki talimata uyma' ilkesiyle çelişiyor ve zayıf modelde ya literal jeton sızıntısı ya boş-sonuç uydurması üretiyor.	Hata/boş durumları serbest metin yerine AracSonucu.durum üzerinden yapısal ilet (zaten .basarisiz var). Modele dönen metni ya tamamen boş bırakıp durum kodundan uygulama tarafında Türkçe kullanıcı cümlesi üret, ya da modele giden metni saf veri + kısa nötr etiketle sınırla (imperative cümle olmadan). Böylece 'çıkı = veri, talimat = kod' ayrımı korunur ve dil çapası zorlanmaz.
P2	Özetlemeyi de tavan +alaka ile sınırla	Ekleme anında 200'e kadar aracın hepsi tek tek on-device modele özetletiliyor (BaglantiServisi.swift:274-280) ama yalnız 6'sı kullanılıyor; bu, yavaş cihazda dakikalarca süren gereksiz arka plan çıkarımı ve pil maliyeti.	tanimlarilceAktar'da önce ucuz metin sinyaliyle (ad/açıklama) muhtemel kullanılacak ilk N aracı (ör. tavanın birkaç katı) seç, yalnız onları modele özetlet; kalanları 'kaba' kırpma ile bırak ve gerçekten oturuma girerlerse tembel olarak sonra özetle. Böylece ekleme anı hızlı kalır, model çıkarımı yalnız görünme ihtimali olan araçlara harcanır.

A.5 Beceri/guidance enjeksiyonu (progressive disclosure)

Özet kanaat: Tek-beceri progressive disclosure ve istem-sonu enjeksiyonu doğru içgüdüler; ama 700 karakter kör-kesmesi hayati örnek/ kuralları atması, kelime-sınırsız alt-dizge eşleşmesinin yanlış beceri yüklemesi ve enjeksiyonun profil/araç setinden kopuk olması, tasarımı küçük model için güvenilmez kılıyor.

Mevcut durum. Beceri katmanı Claude'un SKILL.md mantığını taklit eder: **Beceriler/*.md** frontmatter (ad + tetikler) + gövde olarak yüklenir (BeceriDeposu.swift:97-131), kullanıcı becerileri SwiftData'da tutulur (KullaniciBecerisi.swift). Her turda mesaja EN İYİ uyan TEK beceri seçilir (BeceriDeposu.swift:64-74) ve o oturuma BİR KEZ, sistem talimatına değil o turun istemine iliştilir (ModelServisi.swift:382-389, 791). Enjeksiyon

<guidance name=...> çiti + "asla anma/özetleme" satırıyla sarılır ve gövde 700 karakterle sınırlanır (BeceriDeposu.swift:78-93); kullanıcı becerisi gövdesi ayrıca 500 karakterle sınırlıdır (KullanıcıBeceri.swift:16). Eşleşme puanı, eşleşen tetiklerin UZUNLUKLARININ toplamıdır (adet değil), böylece "tablo olarak" gibi özgül ifade "tablo"yu yener (BeceriDeposu.swift:64-74). Eşleşme `s.contains($1)` ile ham alt-dizge taramasıdır (BeceriDeposu.swift:68). Profil/araç seçimi ise TAMAMEN AYRI bir anahtar-kelime sistemiyle yapılır (niyetProfili + gundeliklzleri/belgelzleri/aramalzleri, ModelServisi.swift:527-657) ve beceri enjeksiyonu bu profilden bağımsız çalışır.

Güçlü yanlar:

- Progressive disclosure temel kararı doğru: 10 becerinin hepsini talimata gömmek yerine yalnız 1 beceri, 1 kez enjekte etmek 4096 pencere için doğru içgüdü (BeceriDeposu.swift:5-9, ModelServisi.swift:344-350).
- Beceri kılavuzunu sabit talimata değil o turun istemine ve sorunun HEMEN ÖNÜNE koymak, küçük modelde son-blok ağırlığından yararlanan bilinçli ve ölçüme dayalı bir tercih (ModelServisi.swift:346-367).
- Uzunluk-ağırlıklı puanlama (adet değil) gerçek bir belirsizliği çözüyor: 'bunu tablo olarak göster' → belge-oku, 'tablo yap' → belge-olustur; gerekçe kodda açık (BeceriDeposu.swift:57-63).
- Beceri gövdeleri emir kipinde, aracı ADIYLA anıp somut `tool(arg)` örneği gösteriyor (kod.md:17, web-sayfa.md:17) — bu, modeli 'anlatmaya' değil araç çağdırmaya iten doğru biçim.
- Kullanıcının kendi becerisinde sert 500 karakter tavanı UI'da canlı sayaçla dayatılıyor (BeceriPanosu.swift:250,341-345), böylece kullanıcı bütçeyi sessizce şişiremiyor.

Zayıflıklar:

Zayıflık Kanıt

Küçük-model etkisi

[KRİTİK] 700 karakter kesme, en uzun 5 becerinin ÖRNEK ve KURAL bölümlemini (halüsinasyon çitleri) sondan atıyor	BeceriDeposu.swift:78-85 gövdeyi son satır sınırında 700'e kesiyor. Ölçüm: kod.md gövdesi 1069 → 690 karakter tutulup ## Example (kod_calistir örneği) VE ## Rules ("Never claim you ran code without a successful tool call") tamamen düşüyor; web-sayfa.md 1248 → 683, ## Example + tüm ## Rules (tek çağrı, düzenleme akışı, doğrulama-hatası retry) düşüyor; belge-olustur.md 1059 → 599, "NEVER compute totals yourself" =SUM kuralı + kaynakRef kuralı düşüyor.	3B model için somut tool(args) örneği doğru araç çağrısının EN GÜÇLÜ sinyalidir; kesme tam da onu ve 'aracı çağırmadan yaptım deme / toplamı kendin hesaplama' anti-halüsinasyon kurallarını atıyor. Sonuç: kod becerisinde model sonucu uydurabilir, web-sayfa'da ham HTML yazmaya kayabilir, excel'de toplamı kafadan hesaplayabilir — becerinin çözmeye çalıştığı hataların tam kendisi.
[Yüksek] Alt-dizge eşleşmesi (kelime sınırı yok) sık kullanılan Türkçe kelimelerde yanlış beceri enjekte ediyor	BeceriDeposu.swift:68 s.contains(\$1); kısa tetikler: arama.md 'ara', 'bul', belge-oku.md 'oku'. Test: 'hava bugün bulutlu mu' → 'bul' ∈ 'bulutlu' → arama becerisi; 'dokunma' → 'oku' ∈ 'dokun' → belge-oku; 'onlara/tara/kara/okul' hepsi tetikliyor.	Bir hava durumu sorusu (başka tetik yoksa skor 3 ile) arama becerisini kazandırıyor; bu beceri modele Spotlight kullan ve 'Cihazında bulamadım' de diyor (arama.md:9-12) — soruyu yanlış araca yönlendirip yanlış 'cihazda yok' cevabına itiyor. Küçük model zaten kırılğan; yanlış kılavuz onu doğrudan yanlış davranışa kilitler.

[Yüksek] Beceri enjeksiyonunu aktif profile/ araç setine bağlı DEĞİL: kılavuz, oturumda a BULUNMAYAN bir aracı çağırma yolum emrede biliyor	BeceriDeposu.eslesen (BeceriDeposu.swift:64) profilden habersiz. Profiller araçları kısıtlar: aramaAraclar yalnız web_arama+hesap+zaman verir, kod/belge araçları YOK (ModelServisi.swift:320-325). Örn 'python ile dolar kurunu hesapla' → 'dolar' aramaları'nde (ModelServisi.swift:632) profil=.arama; skill matcher 'python' ile kod becerisini enjekte edip "Call kod_calistir" der ama kod_calistir masada yok.	Model, oturumun araç şemasında olmayan bir araca çağrı yapmaya itiliyor; bu ya başarısız/uydurma bir çağrıya ya da 'anlatma' moduna düşürür. Küçük modelde araç adı ile şema uyumsuzluğu en kırılğan noktalardan biri.
[Orta] İki paralel, elle senkronize edilen anahtar-kelime sistemi (beceri tetikleri vs profil izleri) birbirinden kayabiliyor	Profil seçimi gundeliklari/ belgelzleri/aramalari/baglantiizleri ile (ModelServisi.swift:596-657), beceri seçimi .md tetikler ile (BeceriDeposu.swift:64) — iki ayrı liste, ortak kaynak yok. Örn belgelzleri 'sayfa/site/html' içerirken (ModelServisi.swift:613) web-sayfa.md tetikleri farklı bir kelime kümesi tutuyor (web-sayfa.md:3).	Bir listeye kelime eklenip diğerine eklenmediğinde profil ile enjekte edilen beceri çelişebilir; model doğru araç setiyle ama yanlış kılavuzla (veya tersi) kalır. Bakım hatası doğrudan küçük-model davranışına sızar.

[Orta] Beceri oturuma bir kez enjekte edilip kilitleniyor; gövde bağlam penceresinden kaysa da yeniden enjekte edilmiyor	enjekteBeceriler seti (ModelServisi.swift:351) turda beceriyi ekler (382-383) ve yalnızca oturumKur içinde temizlenir (415). Uzun tek-konulu bir oturumda beceri 1. turda enjekte edilip transcript büyüdükçe üretim noktasından uzaklaşır; özetleme (butceKontrol, ModelServisi.swift:907) tetiklenene kadar tekrar enjekte edilmez ve özet metni guidance'ı taşımaz.	Küçük modelde etkinin yakınlıkla (recency) düştüğü bilinir; 8. turda hâlâ kod yazan kullanıcıda kod kılavuzu artık uzakta kaldığı için model kurallara uymayı bırakabilir, ama sistem 'zaten enjekte ettim' diye yeniden vermez.
--	---	---

Öneriler:

Öncelik	Başlık	Gerekçe	Taslak yaklaşım
P0	Enjeksiyon bütçesini kör sondan-kesmeden 'çekird ek- önce' modele çevir	700 kesme, küçük model için en değerli iki parçayı (somut araç-çağrı örneği + anti-halüsinasyon kuralı) tam da attığı için beceri kendi amacına ters düşüyor (kanıt: kod.md/web-sayfa.md/belge-olustur.md ölçümleri).	İki yol: (a) Her .md gövdesini 'modele giden çekirdek' ($\leq \sim 400$ karakter: tek satır amaç + tek tool(args) örneği + en kritik 1-2 kural) ve 'yalnızca insan referansı olan kuyruk' olarak frontmatter'da ayır; enjeksiyon yalnız çekirdeği alsın, kuyruğu asla. (b) Bunu yapmadan önce yazım kuralını tersine çevir: örnek ve sert kurallar gövdenin BAŞINA, ayrıntı akış anlatımı sona; böylece kesme olsa bile hayati kısım hayatta kalır. Ayrıca kesme gerçekleştiğinde bunu bir uyarı/log ile görünür kıl — bugün sessiz.

P0	Tetik eşleşmesini kelime-sınırlı hale getir, çok kısa tetikleri exact-only işaretle	Alt-dizge taraması 'bulutlu'→arama, 'dokun'→belge-oku gibi sık kelimelerde yanlış beceri yükleyip küçük modeli yanlış araca kilitliyor (BeceriDeposu.swift:68).	Soru ve tetikleri kelime/token sınırında karşılaştır (harf-dışı ayraçlarla böl, tam kelime veya çok-kelimeli ifade eşleşmesi ara). Tek kelimelik ≤3-4 karakter tetikler ('ara', 'bul', 'oku') yalnız tam-kelime olarak eşleşsin; ifade tetikleri ('tablo olarak') alt-dizi olarak kalsın. Uzunluk-ağırlıklı puanlama korunur ama artık gürültüsüz girdiler üzerinden çalışır.
P1	Beceri enjeksiyonunu aktif profilin araç setiyle çaprazla	Kılavuz, oturumda bulunmayan aracı ('Call kod_calistir') çağırmayı emrederek küçük modeli şema-dışı çağrıya veya anlatmaya itiyor (aramaAraclar ModelServisi.swift:320 vs eslesen bağımsız).	Her beceriye, ilgili olduğu araç adlarını (frontmatter'da araclar: kod_calistir gibi) ekle. eslesen çağırısı seçilmiş profilin araç kümesini parametre alsın; becerinin araçlarından en az biri o kümede yoksa o beceriyi enjekte etme (veya profile uygun bir alternatif ara). Böylece guidance her zaman masadaki araçlardan bahseder.
P1	Profil yönlendirilmesi ile beceri eşleşmesini tek bir tetik kaynağından üret	İki ayrı elle-bakımlı liste (Izler vs tetikler) kaçınılmaz olarak kayar; kayma doğrudan yanlış profil+kılavuz kombinasyonu olarak küçük modele yansır.	Beceri .md tetiklerini tek gerçek kaynağı yap; her beceriyi bir profile etiketle (frontmatter profil:). niyetProfili, ayrı Izler listeleri yerine 'eşleşen becerinin profili'ni sinyal olarak kullansın (veya en azından onunla oylaşsın). Böylece 'hangi beceri' ve 'hangi araçlar' kararı aynı veriden çıkar, drift imkânsızlaşır.

P2	Bir-kez-enjekte kilidini 'pencer eden kayınca yeniden enjekte 'ye gevşet	Uzun tek-konulu oturumda kılavuz bağlamdan uzaklaşıp etkisini yitirebilirken sistem yeniden vermiyor (enjekteBeceriler ModelServisi.swift:351,415).	enjekteBeceriler'i kalıcı kilit yerine 'son enjeksiyon turu/token konumu' olarak tut. Aynı beceri tekrar eşleşiyor ve önceki enjeksiyondan bu yana N tur (veya tahmini token mesafesi) geçtiyse yeniden enjekte et. Böylece tekrarlı bütçe israfı olmadan kılavuz üretim noktasına yakın kalır.
----	--	---	---

A.6 Yapısal çıktı & ayrıştırma kırılğanlığı

Özet kanaat: Deterministik güvenceler (kod sandbox, HTML doğrulama, roundtrip testi) güçlü; ancak tablo ayrıştırma tek katı biçime bağlı ve hata yolunda sessiz kayıp/çöp-dosya üretiyor — küçük modelin en olası sapmalarında (eksik ayraç, dış-pipe'sız, kesik akış) yapı görünmez şekilde bozuluyor.

Mevcut durum. Model iki kanaldan yapısal çıktı verir: (a) sohbet metnine gömülü markdown tablo, (b) araç argümanı olarak markdown (belge_olustur/duzenle icerik). Her ikisi de aynı ayrıştırıcıya bağlanır: Tablo.markdownTabloları, satırın "\\|\" ile başlamasını, bir sonraki satırın "\\|\" ile başlayıp \"-\" içermesini (ayraç) ve en az bir veri satırını şart koşar (Tablo.swift:56-84). Sohbet tablo yalnızca üretim BİTİNCE render edilir; akış sırasında düz metin gösterilir, böylece yarım tablo titremez (KetumYaniti.swift:147-164). Kod çıktısı deterministiktir: print yoksa son ifade değeri çıktı sayılır (KodMotoru.swift:96-98) ve tur başına en fazla 2 çalıştırma, sonra error_final (KodCalistirAraci.swift:62-68). HTML tarafında model yalnız markdown üretir, iskele+stil gömülüdür ve üretilen sayfa ekran-dışı WKWebView'de doğrulanır; geçmezse dosya silinip modele "simplify and try once more" döner (HtmlMotor.swift:5-12, BelgeOlusturAraci.swift:65-77). Modele format baskısı açıktır: "To show a table, output the markdown table rows (\\| ... \\|) themselves — a sentence instead of the rows is a failure" (Yonlendirici.swift:40-41).

Güçlü yanlar:

- Akış sırasında tablo render EDİLMEZ, düz metin gösterilir (KetumYaniti.swift:147-152) — küçük modelin token-token akan yarım tablosu ekranda titremez, snap sonda bir kez olur.

- Kod yürütmede halüsinasyon yüzeyi deterministik olarak kapatılmış: model sonucu iddia edemez, araç çalıştırır; print yoksa son ifade değeri fallback'tir (KodMotoru.swift:96-98); deneme sayacı motoru 3. çağrıda hiç çalıştırmaz (KodCalistirAraci.swift:60-68).
- HTML iskelesi+CSS uygulamada gömülü; model şablonu hiç görmez, 4096 pencere içerik için kalır (HtmlMotor.swift:5-12). Üretilen sayfa deterministik olarak doğrulanır, bozuksa kullanıcıya sunulmaz (SayfaDogrulayici.swift, BelgeOlusturAraci.swift:65-77).
- Ragged (eksik hücreli) satır toleransı var: hem markdown üretiminde hem gösterimde hücre sayısı başlığa göre paddinglenir (Tablo.swift:103-105, SohbetTablo.swift:30, HtmlMotor.swift:190-192).
- Tablo roundtrip'i (markdown ↔ Tablo) simetrik tasarlanmış ve OtoTest'te doğrulanıyor (OtoTest.swift:87-94); belge_oku tabloyu 30 satıra kırpıp modele markdown döndürerek bütçeyi koruyor (BelgeOkuAraci.swift:43, Tablo.swift:47-52).
- belge_olustur bicim alanı serbest metin ama BelgeBicimi(kullaniciMetni:) ile toleranslı eşlenir; 'python' denilse de kod JS'te çözülür (KodCalistirAraci.swift:70) — enum ıskalaması çökmeye dönmez.

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
[Yüksek] Tablo algılama kapısı çok katı; yaygın küçük-model sapmalar ı ham pipe metnine düşer	KetumYaniti.swift:190 `if s.hasPrefix("\`	"), sonraki.hasPrefix("\`

[Yüksek] Excel düzenleme/ üretiminde bozuk tablo → tek sütunlu çöp xlsx (sessiz yapı kaybı)	BelgeDuzenleAraci.swift:46-50 <code>tablo = markdownDan(yeniIcerik)</code> ; nil ise <code>govde: tablo == nil ? yeniIcerik : nil</code>. ExcelMotor.swift:25-28: tablo yoksa govde her satırı tek sütuna ('Metin') döker.	Model düzenlenmiş tabloyu dış pipe'sız/ayraçsız üretirse markdownDan nil döner ve ham markdown metni tek sütunlu bir xlsx'e dökülür. Kullanıcı 'düzenlendi' çipini görür, dosyayı açınca sütunları ezilmiş bir tablo bulur — başarı sinyali yanlış.
[Orta] Yarım/ bozuk tablo sessiz KAYIP (ne tablo ne metin)	KetumYaniti.swift:195-202: pipe ile başlayan satırlar <code>tabloSatire</code> toplanır, <code>if let tb = markdownDan(...)</code> başarısızsa hiçbir şey eklenmez ve <code>i = j</code> ile satırlar atlanır; else dalı yok.	Üretim ayraç satırından hemen sonra kesilirse (durdurBas kısmı kaydı, SohbetGorunumu.swift:379-400) veya model header+ayraç yazıp veri satırını 'wrote' diye özetlerse, markdownDan nil döner (Tablo.swift:75 en az bir veri satırı şartı) ve toplanan pipe satırları tamamen buharlaşır — kullanıcı eksik bir yanıt görür, uyarı yok.
[Orta] belge_olustur yalnızca İLK tabloyu alır, kalan içeriği düşürür	BelgeOlusturAraci.swift:49-53: <code>Tablo.markdownDan(ic)</code> (yalnız ilk tablo, Tablo.swift:87-89) başarılıysa <code>govde = nil</code> yapılır.	Model 'başlık + tablo + açıklama' ya da iki tablo üretirse, Excel çıktısına yalnız ilk tablo girer; başlık ve ikinci tablo/metin sessizce kaybolur. Küçük model çok-bloklu çıktıyı düzenli üretemediği için bu yaygın.

[Orta] Bozuk- çıktı testi yok; parser sözleşme si yalnız ideal biçimde kilitli	OtoTest.swift:87-94 roundtrip'i Tablo.markdown'ın ürettiği KUSURSUZ markdown ile test eder; eksik ayraç, dış- pipe'sız, ragged, header- only, çok-tablo veya kesik- akış senaryosu hiçbir vakada yok (OtoTestVakalari.swift'te tablo testi 665-667 yalnız web arama tablosu).	Gerçekte modelin ürettiği bozuk biçimler test edilmediği için, ayrıştırıcı gevşetilse/sıkılaştırılsa regresyon fark edilmez; küçük- model çıktısının en kırılgan yüzeyi otomatik güvence dışında.
[Düşük] KodCalist ir 'dil' zorunlu @Genera ble alanı, açıklama sı sadece 'js' ve tamamen yok sayılıyor	KodCalistirAraci.swift:43-44 @Guide(description: "js") var dil: String (opsiyonel değil); değeri KodCalistirAraci.swift:70'te tamamen görmezden gelinir.	Küçük modelin her araç çağrısında üretmesi gereken ama hiç kullanılmayan zorunlu bir alan; şema karmaşıklığını ve üretim/ayrıştırma başarısızlık yüzeyini bedava artırır.

Öneriler:

Ön celi k	Başlık	Gerekçe	Taslak yaklaşım
P0	Toleran slı tek tablo ayrıştır cı + kayıp yerine kurtar ma	Zayıflık 1 ve 2: küçük model tek katı biçimi güvenilir üretemez; kapı sıkı, hata yolunda satırlar sessizce yok oluyor.	markdownTablolari'ni gevşet: (a) ayraç satırını OPSİYONEL yap — ardışık, aynı '\

P1	Excel yolund a güvenli düşüş: bozuk tabloyu tek-sütuna dökme, modele geri ver	Zayıflık 3: bozuk tablo sessizce çöp xlsx üretilmiş yanlış başarı sinyali veriyor.	belge_olustur/belge_duzenle'de bicim.tabloYapisi true iken markdownDan nil dönerse ExcelMotor'a govde dökümü yaptırma; bunun yerine HTML doğrulama desenindeki (BelgeOlusturAraci.swift:65-77) gibi verification_failed benzeri bir hata döndür: modele 'tabloyu \
P1	belge üretimi nde çoklu tablo + çevre metnini koru	Zayıflık 4: ilk tablo dışındaki her şey Excel'de kayboluyor.	belge_olustur'da tek markdownDan yerine markdownTablolari(hepsi) kullan. Excel için: birden çok tablo varsa ya çok sayfalı yaz ya da en büyüğünü seçip diğerlerini not olarak modele bildir. Prose biçimlerde (pdf/word/metin) tabloyu govde AKIŞINDA yerinde bırak (govde=nil yapma), böylece başlık ve açıklama metni de dosyaya girsin. Küçük modelin dağınık çok-bloklu çıktısı veri kaybetmeden yansıtılsın.
P1	Bozuk-çıktı test matrisi ile parser sözleşmesini kilitle	Zayıflık 5: gerçek küçük-model sapmaları hiç test edilmiyor, gevşetme regresyona açık.	OtoTestVakalari'na yeni bir vaka grubu ekle: eksik ayraç, dış-pipe'sız satır, ragged satır, yalnız-başlık, iki ardışık tablo, ayraçtan sonra kesilmiş akış girdilerini elle yaz ve hem KetumYaniti.bloklar hem Tablo.markdownTablolari hem belge_duzenle Excel yolundan geçir; her birinin 'ya doğru tablo ya görünür metin, asla sessiz kayıp' invariyanını doğrula. Bu, P0/P1 gevşetmelerini güvenle yapmanın önkoşulu.
P2	Araç şemalarını küçük modele göre sadeleştir	Zayıflık 6: kullanılmayan zorunlu alanlar üretim yükü ve başarısızlık yüzeyi ekler.	KodCalistir.Arguments.dil alanını kaldır ya da opsiyonel yap (v1'de tek motor JS, değer zaten yok sayılıyor). Genel ilke: @Generable argümanlarında modelin doldurması gereken her zorunlu alan bir üretim riskidir; yalnızca davranışı gerçekten değiştiren alanlar zorunlu kalsın, gerisi opsiyonel/çıkarılsın. Böylece küçük modelin araç-çağrı şeması sığ ve kırılğan-olmayan kalır.

P2	Akış- sonu snap ve raw- pipe fallbac k'i yumuş at	Zayıflık 1'in kullanıcı- deneyimi kuyruğu: final parse başarısızsa ham pipe metni gövdede kalıyor.	KetumYaniti'de final render tablo üretemezse, pipe içeren blokları en azından monospace/pre bloğu olarak çiz (proportional serif içinde hizasız pipe metni yerine). İsteğe bağlı: parse başarısızsa küçük bir 'tablo biçimi tanınmadı' ipucu göster. Amaç, küçük modelin biçim ıskalamasında bile çıktının okunur kalması; sessiz kayıp da çirkin ham metin de son çare olmasın.
----	---	---	--

A.7 Toplu veri offload & 4096 bypass (VeriDeposu + kaynakRef)

Özet kanaat: Offload mimarisi kanal ayrımı ve monotonik referansla deterministik olarak sağlam kurulmuş, ama zincirin küçük-model açısından en kırılgan halkası — referansın turlar arası doğru taşınması — istme bırakılmış ve çözölemeyen ref sessizce boş/eksik belgeyle 'başarı' olarak sonuçlanıyor; bu tek nokta düzeltilmeden desen küçük modelde güvenilir değil.

Mevcut durum. Toplu cihaz/belge verisi modelden geçmeden VeriDeposu üzerinden taşıyor: kaynak araç Tablo'yu **koy** ile saklayıp kısa bir referans ID üretiyor (VeriDeposu.swift:23-28), üretim aracı BelgeOlusturAraci **al(ref)** ile tam tabloyu depodan çekiyor (BelgeOlusturAraci.swift:46). Modele DÖNEN metin ile çip detayındaki HAM çıktı deterministik olarak ayrı: araç sözleşmesi yalnızca **modeleDonen**'i modele geri verir, **hamCikti** çipte kalır (KetumAraci.swift:41, AracSonucu:84-92). Kaynak araçlar modele yalnızca kırkık önizleme + ref veriyor: Takvim ilk 10 etkinliğin özeti + **data_ref** (TakvimAraci.swift:124-148, tetik satırlar.count>1 satır 67), Hatırlatici ilk 10 + **data_ref** (HatırlaticiAraci.swift:322-352), WebArama ≤~300 token liste + **data_ref**, tam URL'siz (WebAramalstemcisi.swift:235-253, 656), MCP/bağlantı çıktısı 800 karakter üstünde son 30 satır + **kaynakRef** (BaglantiServisi.swift:418-445). Ancak belge_oku bu desenin DIŞINDA: okunan içeriği doğrudan modele döküyor — 30 satırlık markdown tablo ya da düz metin, 1500 karaktere kırpılmış (BelgeOkuAraci.swift:43-44). KisiAraci'nın hiç depo kanalı yok, yalnız ilk 5 kişiyi özetliyor (KisiAraci.swift:80-87).

Güçlü yanlar:

- Kanal ayrımı DETERMİNİSTİK ve tek noktada: araç ne dönerse dönsün

modele yalnızca **modeleDonen** gider, **hamCikti** çipte kalır (KetumAraci.swift:41). Model tam toplu veriyi hiçbir zaman görmez; bu, modelin iyi niyetine değil koda dayanıyor.

- Referans üretici monotonik: **temizle()** depoyu boşaltır ama sayacı KASITLI sıfırlamaz (VeriDeposu.swift:40-48). Yeni sohbette 'takvim-1' yeniden üretilip eski bir referansa çarpma / başka sohbetin verisinden belge üretme riski tümünden kapatılmış — küçük modelin eski ref'i hatırlaması hâlinde bile veri karışmaz.
- al()** toleranslı eşleme yapıyor: model ref'e tırnak/boşluk eklerse temizleyip yine buluyor (VeriDeposu.swift:33). Küçük modelin tipik kirliliğinin bir kısmını emiyor.
- Modele giden özetler gerçekten sınırlı: önizlemeler prefix(10)/prefix(5) ile, web özeti satır başına 200 karakter tavanı + toplam $\leq \sim 300$ token ve TAM URL modele gitmiyor (WebAramalstemcisi.swift:232-253) — halüsinasyonlu link ve 'X sonuç bulundu → sayı uydur' yüzeyi bilinçli daraltılmış (yorum 244-249).
- Depoya koyma eşiklenmiş: tek kayıtlı okuma ref üretmiyor (count>1 kapıları), MCP çıktısı 800 karakter altındaysa olduğu gibi geçiyor (BaglantiServisi.swift:420) — gereksiz ref üretilip modelin kafası karıştırılmıyor.

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
[KRİTİK] kaynakRef çözülemezse e SESSİZ başarı: belge boş/ eksik üretilir, modele 'file_create d' döner	BelgeOlusturAraci.swift:46-54 — if let ref = arguments.kaynakRef, let depoTablo = await veriDeposu?.al(ref); al nil dönerse bu dal tümünden atlanır, govde = arguments.icerik (çoğunlukla nil, çünkü model ref kullanınca içeriği boş bırakması söylenmiş: Arguments guide satır 32 + description satır 15) ile devam eder. Ref-miss için HİÇBİR hata dalı yok; akış 78-87'de her hâlükârda file_created başarı metnini döndürür.	Küçük model çok-adımlı ref zincirinde ID'yi kolayca bozar (bkz. sonraki bulgu). Ref eşleşmezse kullanıcı, doğru görünen ama BOŞ ya da yalnız 'icerik'ten (modelin uydurduğu kısmi veri) dolu bir Excel/PDF alır; model 'Hazır' der. Sessiz, teşhisi zor veri kaybı — asistanın en tehlikeli hata sınıfı.

[Yüksek] Referans etiketi tutarsız: modele 'data_ref=' gösteriliyor ama argüman adı 'kaynakRef', MCP tarafı 'kaynakRef=' diyor	TakvimAraci.swift:73 data_ref=\(ref), HatirlaticiAraci.swift:351 data_ref=\(ref), WebAramaAraci.swift:79 data_ref=\(ref) — ama BaglantiServisi.swift:434 kaynakRef=\(ref); belge_olustur argümanının adı ise kaynakRef (BelgeOlusturAraci.swift:33). Model, kaynağa göre 'data_ref' ya da 'kaynakRef' gördükten sonra bunu 'kaynakRef' argümanına eşlemek zorunda.	Zayıf talimat-takipli 3B model isim eşlemesinde kırılıgandır: 'data_ref=takvim-1' metninden yalnız 'takvim-1'i çıkarıp doğru alana koyması gerekir. Sıkça ya tüm 'data_ref=takvim-1' dizesini argümana yazar (al() 'data_ref=' önekini SOYMAZ — VeriDeposu.swift:33 yalnız tırnak/boşluk kırpar → eşleşmez → kritik bulgudaki sessiz başarı) ya da hiç bulamaz.
[Yüksek] Ref zinciri çok-adımlı: model referansı turlar arası 4096 pencerede taşınmalı; önizlemede ki 10 gerçek satır 'kendin yaz' cazibesi yaratıyor	Akış: tur 1 kaynak araç modeleDonen'de özet + ref döner (TakvimAraci.swift:69-74), tur 2 model bu ref'i hatırlayıp belge_olustur.kaynakRef'e geçirmeli. Aynı modeleDonen modele 10 gerçek etkinliği de veriyor (prefix(10), satır 124-126). Talimat 'veriyi kendin yazma' diyor (Yonlendirici.swift:57-58) ama bu yalnızca istem-temelli, koda bağlı değil.	Küçük model önünde duran 10 hazır satırı görünce, ref'i taşıyıp ikinci araç çağırmak yerine bu 10 satırı doğrudan icerik'e markdown tablo olarak yazmaya meyleder; belge_olustur bunu memnuniyetle kabul eder (tabloYapisi dalı, BelgeOlusturAraci.swift:49-53). Sonuç: 200 kayıtlık takvimden 10 satırlık 'tamam görünen' Excel — eksiklik sinyali yok. Offload deseninin varlık nedeni tam da bu senaryoda deviliyor.

[Orta] belge_oku offload desenin DIŞINDA: okunan belge içeriği doğrudan bağlama dökülüyor	BelgeOkuAraci.swift:43-44 — govde = icerik.tablo?.markdownKirpi k(enFazlaSatir:30) ?? icerik.metin; kirpik = govde.count > 1500 ? prefix(1500)+... : govde ve bu doğrudan modeleDonen'e (satır 51). 30 satırlık markdown tablo + 1500 karakter, ref kanalı kullanmadan modele giriyor.	Tek bir belge okuması ~375+ token'ı 4096 bütçenin ~%10'unu tek kalemde yer. 'Belgeyi tablo göster' + 'bir satır ekle' gibi çok turlu düzenlemede pencere hızla dolar; model kırpılmış 1500 karakterden çalışıp belgenin görünmeyen kısmını 'yok sayar' ya da uydurur. Diğer araçlar veriyi depoya koyarken bu araç koymuyor — desen tutarsız.
[Düşük] KisiAraci'nın depo kanalı yok — offload deseni her toplu-veri aracına uygulanma mış	KisiAraci.swift:80-87: ozet = kisiler.prefix(5)..., tam = kisiler.map... çip için ama VeriDeposu'ya koy çağrısı YOK; araçta veriDeposu alanı bile tanımlı değil. Aynı soyad çok kişi döndürebilir, modele yalnız 5'i gider.	'Tüm iş arkadaşlarımı Excel'e dök' türü bir istek kişiler için desteklenmiyor: ref üretilmediğinden model yalnız 5 kişilik özetten belge yazmaya çalışır — sessizce eksik. Pratikte kisi bir 'isimle arama' aracı olduğu için etki dar, ama desen boşluğu belge/dış- veri büyürse tuzağa döner.
[Düşük] Bozuk/ halüsinasyo nlu kaynakRef için test yok — sessiz- başarı yolu regresyon koruması altında değil	OtoTestVakalari.swift yalnız VeriDeposu tablo yapısını (664-667) ve özet bütçesini (650-658) doğruluyor; al()'in eşleşmeyen/önekli ref'te davranışını ya da belge_olustur'un ref-miss dalını sınavan vaka yok (grep 'kaynakRef'/'al(' üretim yolunda boş).	Kritik sessiz-başarı yolu (bulgu 1) hiç kırmızıya düşmediği için birisi 'data_ref=' önek soyma ya da hata dalı eklese bile bir güvence ağı yok; küçük- model regresyonları üretimde fark edilir.

Öneriler:

Ön celi k	Başlık	Gerekçe	Taslak yaklaşım
P0	Ref çözüle mezse ASLA sessiz belge üretme — açık hata dalı + stale önizle meden yazma yı engelle	En tehlikeli hata boş/ eksik belgeyi 'başarı' olarak sunmak (zayıflık 1 ve 3). Küçük model ref'i sık bozar; kod bunu deterministik yakalamalı, istme güvenmemeli .	BelgeOlusturAraci'nda: kaynakRef DOLU ama al() nil dönerse dalı sessizce atlama; verification_failed benzeri bir hata metni döndür ('ref \ (ref) bulunamadı; kaynak aracı yeniden çağır ya da geçerli data_ref ver — belge OLUŞTURULMADI') ve dosya yazma. Ayrıca ref verilmişken icerik 'ten tablo kurmaya izin verme (kısmi-veri cazibesini kapat). İkinci katman: al() içinde 'data_ref='/'kaynakRef=' örneklerini ve # gibi süsleri de kırp (VeriDeposu.swift:33 toleransını genişlet) ki modelin en sık kirliliği yine de eşleşsin.
P0	Tek referan s etiketi ve model için ayrıştır ması kolay, opak bir jeton biçimi	İki farklı etiket (data_ref vs kaynakRef) + argüman adı uyumsuzluğu , zayıf modelde eşleme hatasının başlıca kaynağı (zayıflık 2).	Tüm kaynak araçların modele döndürdüğü etiketi TEK biçime sabitle — argüman adıyla birebir aynı: örn. her yerde kaynakRef=takvim-1 . Daha iyisi jetonu kendi içinde tanınır ve serbest metinden ayrık kıl (örn. <ref:takvim-1> ya da tek-kelime opak ref_a1b2), böylece model 'sadece şu jetonu geçir' talimatını daha güvenilir izler. Beceri kılavuzunda ve Yonlendirici'de aynı sözcüğü kullan; şu an 'data_ref' metinlerde, 'kaynakRef' argümanda — birini seç.

P1	Export niyetin de çok-adımlı zinciri kısalt: ref emitildiğinde önizlemeyi bastır ya da tek-adıma indir	Zincir turlar arası olduğu ve önizlemede 10 gerçek satır durduğu için küçük model ref'i atlayıp kısmi veri yazıyor (zayıflık 3). Cazibeyi kaldırmak istemden daha güçlü.	İki seçenek: (a) Kaynak araç ref üretirken modele giden önizlemeyi minimuma indir — sayı + 'tam liste hazır, kaynakRef=X' ver, 10 satırı çıkarma (yalnız çıpte kalsın); modelin kopyalayacak veri kalmayınca ref'i taşımak tek yol olur. (b) Daha köklü: 'export' desenini tek araç çağrısına indir — kaynak aracın kendisi bir dosyaya_dok=excel argümanı alsın, veriyi hiç modele döndürmeden doğrudan üretim motoruna verip dosyayı yazsın. Böylece ref küçük modelin belleğinden hiç geçmez.
P2	belge_oku'yu da offload kanalına al: tam içerik depoya, modele sınırlı özet + ref	belge_oku desenin tek istisnası ve tek turda ~375+ token döküyor (zayıflık 4); çok turlu 'oku→tablo göster→satır ekle' akışında pencereyi doldurur.	Okunan belgenin Tablo/metin tamamını VeriDeposu'ya koy, modele yalnız ilk ~10 satır/ kısa özet + kaynakRef ver — diğer araçlarla aynı sözleşme. 'Tablo olarak göster' ve 'düzenle' akışları içeriği bağlamdan değil depodan çeksın. Böylece belge_oku, belge_düzenle ve belge_olustur aynı ref semantiğini paylaşır ve okuma-düzenleme döngüsü 4096'yı yormaz.

P2	Ref-miss ve önekli-ref için deterministik test vakaları ekle	Kritik sessiz-başarı yolu regresyon koruması altında değil (zayıflık 6); küçük-model davranışı istemle ayarlandığı için kod güvencesi şart.	OtoTestVakalari'na: (1) belge_olustur'a var olmayan ref → dosya yazılmadığını ve hata metni döndüğünü doğrula; (2) <code>al("data_ref=takvim-1")</code> gibi önekli/tırnaklı ref'in doğru tabloyu bulunduğunu doğrula; (3) temizle() sonrası eski ref'in yeni tabloya ÇARPMADIĞINI (monotonik sayaç) doğrula. Bu üç vaka offload sözleşmesinin küçük-model kenar durumlarını kilitler.
----	--	---	--

A.8 Hata/retry/iptal semantiği (küçük-model uyumu)

Özet kanaat: İptal ve YEREL yan-etki retry semantiği bayrak-tabanlı ve küçük-modele göre sağlam tasarlanmış; ama uzak (MCP) yazmaların retry korumasının tamamen dışında kalması ve hata kurtarmanın tur-iptaline kör olması, kırılgan 3B modelde çift dış yan etki ve görünmez yarış üretebilecek kritik boşluklar.

Mevcut durum. Bir turun sonucu METİNDEN değil açık bayraklardan okunur: `YanitSonucu` (`hataMi`, `tekrarDenenebilir`, `geciciMi`) servis tarafından doldurulur, UI metin karşılaştırmaz (ModelServisi.swift:41-53; SohbetGorunumu.swift:358-364). Retry güvenliği tek bir yapışkan boole ile korunur: bir araç `.yazildi` bildirince `AracYurutucu.dunyaDegisti` true olur (AracYurutucu.swift:140,163) ve `hataKurtar` en başta bunu görürse retry'ı tümünden engelleyip yalnız hata balonu döndürür (ModelServisi.swift:817-821); yan etki yoksa taşma (`exceededContextWindowSize`) özetleyip bir kez, diğer geçici hatalar taze oturumla bir kez yeniden dener (ModelServisi.swift:831-853). Guardrail ve dil hatalarında retry yoktur (ModelServisi.swift:822-830). İptal hata sayılmaz: `akisYut` içindeki akış ayrı bir Task'ta koşar, `durdur()` onu iptal eder, `CancellationError` yakalanıp o ana kadarki `akanMetin` normal metin gibi döner (ModelServisi.swift:856-895). İptal yarışı `uretimNo` ile ele alınır: `durdur()` sayacı ilerletir ve `uretiyoru` derhal false yapar; `yanitSonucu` defer'ı `if uretimNo == benimTur` koşuluyla eski turun geç defer'ının yeni turun bayrağını düşürmesini engeller (ModelServisi.swift:689-706,752-757).

Güçlü yanlar:

- Hata tespiti UI'da metin-eşleştirme YERİNE bayrakla yapılıyor; hata dizgisi değişince balon sessizce ölmüyor (ModelServisi.swift:41-53) — küçük modelin değişken çıktısına karşı doğru mimari tercih.
- İptal ile gerçek hata net ayrılmış: iptal **akanMetini** koruyup normal döner, retry tetiklemez (ModelServisi.swift:891-894); kullanıcı 'dur'a basınca yarım yanıt ekranda kalır.
- **uretimNo** + defer guard, uzun turlarda 'dur'a basıp hemen yeni istem gönderme yarışında yeni turun **uretiyor** bayrağını koruyor (ModelServisi.swift:700,757) — küçük-modelin dakikalar süren turlarında pratikte sık karşılaşılan yarış.
- **durdur()** **uretiyoru** DERHAL false yapıyor; akışın sönmesini beklemiyor, böylece gönder düğmesi stop ikonunda donup yeni isteği reddetmiyor (ModelServisi.swift:697-701).
- Yerel yan etki tekrarına karşı retry kilidi doğru: yazan araç çalıştıysa **hataKurtar** ne otomatik retry yapıyor ne de 'yeniden dene' düğmesini sunuyor (**tekrarDenenebilir=false**) (ModelServisi.swift:817-821; SohbetGorunumu.swift:252).

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
[KRİTİK] MCP/ uzak yazma işlemleri retry- güvenliği nin DIŞINDA — çift uzak yan etki riski	MCPAraci.swift:227-235 uzak çağrı için bilinçle durum: .okundu bildiriyor (' .yazildi bu uygulamadaki geri alma/kurtarma mantığını tetikler, o yüzden bilinçli olarak .okundu '). .yazildi dışındaki hiçbir durum dunyaDegisti yi kurmuyor (AracYurutucu.swift:163). Dolayısıyla uzak bir 'issue aç / kayıt oluştur' başarıyla döndükten SONRA model hatası çıkarsa hataKurtar dunyaDegisti==false görüp aynı istemi yeniden gönderir (ModelServisi.swift:817,846-851).	~3B model çok-adımlı araç turlarında kırılğan: uzak aracı çağırır, sonra bozuk JSON/taşma/guardrail üretebilir. O anda otomatik retry SUNUCUYA aynı çağrıyı tekrar gönderir → çift Jira issue, çift kayıt. Retry yine hataya düşerse tekrarDenenebilir=true döndüğü için (ModelServisi.swift:852-853) kullanıcıya 'yeniden dene' düğmesi de sunulur → üçüncü kopya. İlginç asimetri: aynı .okundu , kirletEgerBasarili üzerinden oturumKirli yi kuruyor (MCPAraci.swift:59-67) yani onay-kapısı uzak yazmayı yan etki sayıyor ama retry-kapısı saymıyor.

[Yüksek] hataKurtar iptal/ üretimNo koruması taşıyor — iptal edilmiş tur gizli retry üretimi başlatabilir	durdur() yalnız uretimGorevini (iç akış Task'ı) iptal eder ve uretimNoyu ilerletir (ModelServisi.swift:697-701). Ama dıştaki yanitSonucu/hataKurtar async çağrısının benimTur/uretimNo guard'ı YOK. hataKurtar içindeki retry, yeni bir uretimGorevi kurup tam bir model üretimi daha koşturuyor (ModelServisi.swift:836,849) — hiçbir yerde 'bu tur hâlâ güncel mi' kontrolü yapılmadan.	Akış gerçek bir hata (ör. exceededContextWindowSize) fırlattığı anda kullanıcı 'dur'a basıp yeni istem yazarsa: eski turun hataKurtarı uretiyor=false iken arka planda saniyeler süren yeni bir üretim başlatır, taze oturumu (oturumKur) yeniden kurup yeni turun oturumu/izleri ile yarışır. uretiyor false olduğu için UI bunu göstermez — küçük modelin uzun turunda görünmez pil yakımı ve oturum durumu bozulması.
[Yüksek] Araç→model hata sözleşmesi tamamen İngilizce metin-sentinel'e dayanıyor — 'metinle hata tespiti yapma' ilkesiyle çelişiyor	Araç başarısızlığında modele sabit İngilizce dizge dönüyor: "tool_failed: the action could not be completed..." (KetumAraci.swift:51), uzak hata "remote_tool_error: ..." / "remote_call_failed: ..." / "remote_tool_empty: ..." (AracYurutucu.swift:1092-1099; MCPAraci.swift:241,248). Bunların modele okunup kullanıcı diline çevrilmesi tamamen modele bırakılıyor; bu turun YanitSonucu.hataMisi yine de FALSE (normal tamamlanma, ModelServisi.swift:797-799).	Servis→UI yolu bayrak-tabanlı (iyi) ama araç→model yolu metin-tabanlı. Zayıf talimat-takipli 3B model bu sentinel'i ya olduğu gibi ('tool_failed: ...') İngilizce yankılar (kullanıcı bozuk İngilizce cümle görür, üstelik hataMi=false olduğu için hata balonu/retry düğmesi çıkmaz), ya da tümünden yok sayıp 'ekledim/oluşturdum' diye halüsinasyon yapar. Araç başarısızlığı deterministik olarak yakalanmadığı için hata yüzeyi tümüyle model prozasına emanet.

[Orta] dunyaDegisti tek ve kaba bir boole — retry güvenliği araç durum-taksono misinden türetiliyor ama zorlanmıyor	Yan etki yalnız guncelle(... durum: .yazildi ...) çağrısından türetiliyor (AracYurutucu.swift:163). Bir yazma yolunun .yazildi yerine başka durum bildirmesi (MCPAraci.swift:232 örneği bunun kasıtlı hâli) retry-güvenliğini sessizce devre dışı bırakır. Ayrıca hataKurtar retry'ı yeniTur(yanEtkiyiUnut:false) ile izleri temizliyor (ModelServisi.swift:833,846), retry başarılıysa dönen yurutucu.izler yalnız retry'ın çiplerini taşır — ilk denemenin çipleri kaybolur.	Retry güvenlik sınırı, her aracın doğru durum bildirmesine güveniyor; tek bir yanlış durum yeni bir araçta çift yan etki açar. Küçük modelde araç zaten sık başarısız olup retry'a düştüğü için bu kör nokta yüksek frekansla vurulur; ayrıca kaybolan ilk-deneme çipleri kullanıcının 'ne yapıldı' şeffaflığını (seyir) eksiltir.
[Orta] Modelin 'yaptım' iddiası ile gerçek araç çağrısı arasında deterministik çapraz kontrol yok	Talimat 'yalnızca aracı gerçekten çağırdıysan yaptığını söyle' diyor (Yonlendirici.swift:39) ama bu tümüyle talimat; dunyaDegisti/izler gerçek yazmaları biliyor olsa da yanıt metniyle karşılaştırılmıyor. YanıtSonucu yalnız yurutucu.izleri iliştiyor (ModelServisi.swift:799), 'iddia edildi ama çağrı yok' durumunu yakalayan bir kapı yok.	Halüsinasyona yatkın 3B model hiç araç çağırmadan 'etkinliği takvimine ekledim' diyebilir; sistem elinde gerçek sinyal (dunyaDegisti=false, ilgili ikon içermeyen izler) olmasına rağmen bunu ne bir uyarıya ne de bir düzeltmeye çeviriyor — sessiz yanlış-onay, kullanıcı için en pahalı hata türü.

Öneriler:

Öncelik	Başlık	Gerekçe	Taslak yaklaşım
---------	--------	---------	-----------------

P0	Uzak yan etki için ayrı bir 'geri-alınama z dış etki' bayrağı ekleyip retry kapısını genişlet	Bugün uzak yazmalar .okundu bildirildiği için dunyaDegisti false kalıyor ve otomatik retry sunucuya çift çağrı gönderiyor (MCPAraci.swift:227-235; ModelServisi.swift:846-851). Yerel yazmalar korunuyor, uzak yazmalar korunmuyor — bu en pahalı küçük-model hatası.	AracYurutucu'ya dunyaDegisti nin yanına disEtki0lusabilir gibi ikinci bir yapışkan bayrak koy; MCPAraci uzak çağrı BAŞARIYLA dönünce (idempotent olduğu bilinmedikçe) bunu kurar. hataKurtar ın en başındaki guard'ı <code>`dunyaDegisti \</code>
P0	hataKurtar ı tur kimliğine bağla; iptal edilmiş turun kurtarması hiç koşmasın	hataKurtar benimTur/uretimNo görmeden tam bir retry üretimi başlatabiliyor (ModelServisi.swift:836,849); 'dur' + yeni istem senaryosunda görünmez, oturum-bozan bir üretim yarışı doğuyor.	yanitSonucu , benimTuru hataKurtara geçirsin; kurtarma her retry'dan (akisYut) ÖNCE guard uretimNo == benimTur else { return YanitSonucu(hataMi:true, tekrarDenenebilir:true) } kontrolü yapsın. Böylece kullanıcı turu iptal edip yenisini başlattıysa eski turun kurtarması sessizce vazgeçer. Aynı guard butceKontrol/oturumKur yeniden kurmalarının yeni turla yarışmasını da engeller.

P1	Araç başarısızlığını metin-sentinel yerine yapısal sonuca bağla	'Hatayı metinle tespit etme' ilkesi servis→UI'da uygulanıyor ama araç→model yolunda İngilizce sentinel dizgelerine (KetumAraci.swift:51; AracYurutucu.swift:10 92-1099) güveniliyor; zayıf model bunları yankılıyor ya da yok sayıyor, hataMi yine false kalıyor.	Araç başarısızlığını yürütücüde deterministik say: guncelle(... durum: .basarisiz ...) bir 'bu turda araç patladı' bayrağı kursun. YanitSonucu tamamlanınca, model çıktısı boş/çelişkili değilse bile, en azından sentinel dizgelerinin çıktıya sızıp sızmadığını deterministik denetleyip (model prozasına değil, bilinen sabit önekle) hataMi yi işaretlesin; ya da başarısız araç sonrası yanıtı 'araç patladı' bayrağına göre hata balonuna çevir. Modele giden sentinel korunabilir ama nihai UI kararı artık bayraktan gelir, model yorumundan değil.
P1	Yazma araçları nda durum-taksonomisini zorunlu ve tek-yerden türet	Retry güvenliği her aracın doğru .yazildi bildirmesine güveniyor ama bunu zorlayan bir mekanizma yok (AracYurutucu.swift:1 63); tek bir yanlış durum sessizce çift yan etki açar — MCPAraci örneği bunun bilinçli hâli.	AracSonucu'nda 'bu sonuç yan etki oluşturdu mu' bilgisini durum enum'undan ayrı, açık bir alan (yanEtkiTipi: yok/yerel/uzak) olarak modelle; dunyaDegisti/disEtkiOlusabilir bu alandan türetilsin, .yazildi görsel çipe kalsın. Böylece 'çip rengi' ile 'retry güvenliği' iki ayrı eksen olur ve MCPAraci uzak yazmayı görsel olarak .okundu gösterirken retry-güvenliğini doğru bildirebilir.
P2	Retry sonrası ilk denemenin çiplerini /izlerini koru	hataKurtar retry'ı yeniTur(yanEtkiyi Unut:false) ile izleri temizliyor (ModelServisi.swift:8 46); başarılı retry'da dönen izler yalnız ikinci denemeyi taşıyor, ilk denemede kullanıcının gördüğü şeffaflık kayboluyor.	Retry öncesi mevcut izleri bir kenara alıp retry sonrası birleşik listeyi (ilk deneme + retry) YanitSonucu.izlere ver; ya da seyir katmanında ilk denemenin adımlarını 'yeniden denendi' başlığı altında koru. Küçük modelde retry sık olduğu için bu görünürlük kaybı sistematik.

A.9 Değerlendirme & öz-test kapsamı

Özet kanaat: Deterministik çitler örnek düzeyde otomatik test ediliyor ama küçük-modelin asıl kırılğan davranışı (dil çapası, argüman doğruluğu, format uyumu) ya elle-gözle bırakılmış ya da hiçbir kapiya bağlanmamış tek-örneklemlili koşulara terk edilmiş — yani öz-test, modelin en riskli yüzeyini regresyona karşı fiilen korumuyor.

Mevcut durum. Öz-test üç ayrı DEBUG komut-satırı koşucusuna bölünmüş, hepsi elle tetikleniyor (ketumApp.swift:16-24: `--ototest`, `--test`, `--dil`). (1) OtoTest.swift + OtoTestVakalari.swift model/ağ GEREKTİRMEYEN saf iddia takımı: OtoTestDefteri hata sayacıyla gerçek assert üretir (OtoTestVakalari.swift:39-53), hafıza filtrelerini, tekilleştirmeyi, enjeksiyon bütçesini, onay kapısını ve ağ tekeli statik tarar. (2) Degerlendirme.swift GERÇEK on-device modeli ~28 sohbet vakası + hafıza/sevir/web bloklarıyla koşar; skoru çip ikonundan okur (Degerlendirme.swift:124 `izler.map(\.ikon)`) — yani modelin araç seçimini fiilen sınar, ama sonucu yalnız Caches altındaki bir metin dosyasına yazar (satır 190-192), hiçbir eşik/exit-code yok. (3) DilTesti.swift 9 dil × 6 vakayı gerçek modelde koşar ama yalnız araç ikonunu sayar; yanıt dilini "gözle" bırakır (DilTesti.swift:113-114). Projede XCTest hedefi, Cl dosyası ve sampling/temperature kontrolü hiç yok (grep boş; ModelServisi.swift:867 `streamResponse(to: soru)` seçeneksiz).

Güçlü yanlar:

- Deterministik güvenceler cömertçe ve gerçekten assert ediliyor: OtoTestVakalari hafıza filtreleri, tekilleştirme, çağrı başına 2 not / 50 tavan, enjeksiyon bütçesi ve onay kapısını saf-fonksiyon üzerinde sınıyor (OtoTestVakalari.swift:96-205, 673-762) — bunlar küçük modeli çevreleyen 'model olmadan da tutması gereken' çitler ve gerçek pass/fail üretiyorlar.
- Küçük-model gerçekliğine uygun kabul ölçütü: hafıza bloğu 'yanlış pozitif = fail' ile 'kaçırma = kalite uyarısı' nı ayırıyor (Degerlendirme.swift:203-231, 321-344); bir doğru notu kaçırmak koşuyu düşürmüyor, uydurmak düşürüyor.
- Halüsinasyon yüzeyi doğrudan hedefleniyor: dünya-bilgi/hava vakalarında yanıtIcermemeli 'Paris'/'derece' (Degerlendirme.swift:39-40), web-uydurumama'da dürüstlük kalıbı aranıyor (satır 520-533).
- Araç doğrulaması modelin anlatısına değil aracın kendi izine (AracIzi/çip) dayanıyor (AracIzi.swift:29-53); model çip metnini uyduramadığı

için 'araç çağrıldı mı' sinyali güvenilir.

- Token bütçesi statik olarak kilitleniyor: beceri + hafıza en kötü toplamının ~1600 karakterde kaldığı assert ediliyor (OtoTestVakalari.swift:296-303) — 4096 pencereyi doğrudan koruyan bir ölçüm.
- Ağ tekeli statik taraması (OtoTestVakalari.swift:773-823) 'cihazdan ne çıkıyor' sorusunu regresyona karşı kilitliyor; kaynak ağacı okunamazsa sessiz geçmiyor, açıkça fail veriyor.

Zayıflıklar:

Zayıflık	Kanıt	Küçük-model etkisi
[KRİTİK] Gerçek-model davranış testleri hiçbir otomatik kapıya bağlı değil; elle çalıştır-ve-gözle	Degerlendirme.swift:190-192 sonucu dosyaya yazıp <code>print("EVAL bitti")</code> diyor, eşik/exit yok; DilTesti.swift:126 aynı; ketumApp.swift:19-23 yalnız DEBUG komut argümanı ile tetikleniyor; projede XCTest hedefi/CI yok (find boş).	Bu tam olarak memoide anlatılan 'mekanizma kurulur ama üretime bağlanmaz' regresyon deseni. Araç seçimi, dil çapası veya format kuralı bir prompt değişikliğiyle gerilerse hiçbir build/CI kırılmıyor; birinin <code>--test</code> koşmayı hatırlaması, dosyayı açması ve X satırlarını gözle ayıklaması gerekiyor. Zaten kırılabilir olan küçük-model davranışında sessiz gerileme kaçınılmaz.
[Yüksek] Gerçek-model vakaları deterministik değil: vaka başına tek koşu, sampling kontrolü yok	ModelServisi.swift:867 <code>oturum.streamResponse(to:soru)</code> seçeneksiz çağrılıyor; kod tabanında GenerationOptions/temperature/seed hiç yok (grep boş). Degerlendirme.swift:119-123 her vakayı tek kez koşuyor, tekrar/çoğunluk oyu yok.	~3B düşük-bit model varsayılan sıcaklıkta stokastik; aynı vaka bir koşuda ✓, diğerinde X olur. Tek örneklemliler, eşiksiz skor gürültülü bir sinyal üretir — gerçek gerileme ile örneklem gürültüsü ayrışamaz, dolayısıyla eval'a güvenilir karar verilemez.

[Yüksek] Dil çapası (yanıt dili) otomatik ölçülmüyor — yalnız gözle	DilTesti.swift:113-115 toolOk yalnız ikonlar.contains{ \$0.hasPrefix(v.ikon) }; başlıktaki yorum (satır 7 'yanıtın o dilde olup olmadığı log'dan gözle') ve satır 114 yorumu ('yanıt dilini gözlüyoruz') dil kontrolünün assert edilmediğini itiraf ediyor.	Küçük modelin en tipik hatası dil kayması: araç çıktısı/istem İngilizceyken yanıt İngilizceye kayar. Talimat 'always reply in the same language' (Yonlendirici.swift:29-32) diyor ama bu kuralın tutup tutmadığını ölçen tek otomatik iddia yok; 9 dilin çapası fiilen test edilmiyor.
[Yüksek] Argüman doğruluğu neredeyse hiç sınanmıyor — sadece 'araç çağrıldı mı'	Degerlendirme.swift:43-45 hesap vakaları yalnız function ikonuna bakıyor, sonucun 1000/135/200 olduğunu doğrulamıyor; takvim-ekle (satır 54) ve hatırlatıcı (57-58) yalnız ikon sayıyor. hamGirdi alanı mevcut (AracIzi.swift:37) ama argüman yalnız web sorgusu için, o da 'ikincil/kaçırma' olarak inceleniyor (Degerlendirme.swift:509-515). Yalnız kod-asal '1060'ı içerik olarak kontrol ediyor (satır 80).	Küçük model aracı doğru seçip yanlış argüman geçirebilir (yüzde hesabını ters kurar, 14:00 yerine 4:00 yazar, yanlış kişi adı). Bu hata sınıfı tamamen görünmez; eval 'yeşil' der ama kullanıcı yanlış toplantı saati alır. Araç seçimi ölçülüyor, araç doğruluğu ölçülüyor.
[Orta] Biçim uyumu (canlı tablo/markdown çıktısı) gerçek modelde doğrulanmıyor	Tablo markdown round-trip'i yalnız STATİK/motor tarafında test ediliyor (OtoTest.swift:86-99); belge-oku/duzenle canlı vakaları yalnız ikon bakıyor (Degerlendirme.swift:74-75). Talimatın en kırılgan kuralı — 'tabloyu göstermek için \	... \

[Düşük] Aşırı zayıf içerik iddiaları ve ölü kod: test niyeti ile gerçeği ayırılmış	Degerlendirme.swift:48 zaman-saat yanıtIcermeli: ":" — iki nokta geçen HER yanıt geçer (ör. 'Saati bilmiyorum: ...' bile). suSaat() satır 544'te tanımlı ama hiçbir yerden çağırılmıyor (grep yalnız tanımı buluyor), oysa saat vakasında kullanılması beklenirdi.	Zayıf iddia sahte-yeşil üretir: model gerçek saati hiç vermese de vaka geçer, yani bu yeteneğin gerilemesi görünmez. Ölü suSaat() saat testinin bir zamanlar sıkıyken gevşetildiğini ve niyetin kodda paslandığını gösteriyor.
--	--	--

Öneriler:

Ön celi k	Başlık	Gerekçe	Taslak yaklaşım
P0	Model-davranış eval'ini eşikli, exit-code'lu otomatik kapıya bağla	Zayıflık #1 (kritik) memoidenin en çok korktuğu regresyon desendir: davranış testi var ama hiçbir şeyi kırmıyor. Küçük modelde araç seçimi/dil/format en olası gerileme noktaları ve bunlar tam da elle-gözle bırakılan katman.	Degerlendirme ve DilTesti'yi 'dosyaya yaz + gözle' yerine net toplam-oran + eşik üreten bir yapıya taşı; başarısızlıkta non-zero exit ver ki bir UI-test hedefi/simülatör koşusu CI'da build kırsın. Gerçek takvim/kişi verisi gerginliğini çözmek için eval'i sabit, enjekte edilmiş SAHTE fikstür üzerinde koştur — hafıza bloğu zaten bellek-içi ModelContainer kullanıyor (Degerlendirme.swift:296-311); aynı deseni takvim/kişi/not için genişleterek deterministik ve CI-güvenli hale getir. Böylece 'yeşil eval' bir insanın hatırlamasına değil kapıya bağlanır.

P0	Nondeterminizmi bastır: en düşük sıcaklık + vaka başına N-koşu çoğunluk oranı	Zayıflık #2: tek örneklemler, seçeneksiz koşu gürültülü sinyal verir; eşik koymadan önce sinyalin kararlı olması gerekir, yoksa CI flaky olur ve devre dışı bırakılır.	Eval koşusunda modele mümkün olan en deterministik sampling'i (en düşük sıcaklık / greedy) GenerationOptions ile geçir; ayrıca her davranış vakasını N kez (ör. 5) koşup geçme ORANINI raporla ve eşiği oran üzerinden tanımla (ör. $\geq 4/5$ geç). Böylece gerçek gerileme örneklem gürültüsünden ayrışır ve 'bu vaka %60 kararlı' gibi eyleme geçirilebilir bir metrik doğar — küçük modelin doğası gereği ikili pass/fail yanlıcıdır.
P1	Dil çapasını cihaz-üstü dil tespitiyle otomatik assert et	Zayıflık #3: 9 dilde araç seçimi ölçülüp asıl kırılma davranış (yanıt dili) gözle bırakılmış; dil kayması küçük modelin imza hatası ve şu an hiç yakalanmıyor.	Her yanıt üstünde ağırsız, cihaz-üstü bir dil tanıyıcı (NaturalLanguage/NLLanguageRecognizer) çalıştır; baskın dil beklenen dile eşleşmiyorsa vakayı düşür. Kritik senaryoyu ayrı bir vaka yap: araç Türkçe soruya yabancı-dilde sonuç döndürsün, yanıt Türkçe kalmalı — talimatın 'tool results may be in another language, restate in user's language' kuralını (Yonlendirici.swift:30-32) fiilen sınavan tek şey bu olur.
P1	Argüman ve sonuç doğruluğunu hamGirdi/hamCikti üzerinden sına	Zayıflık #4: 'araç çağrıldı' ile 'araç doğru argümanla çağrıldı' arasındaki fark küçük modelde en pahalı hata sınıfı (yanlış saat, ters yüzde) ve şu an tamamen görünmez.	Var olan Araclzi.hamGirdi/hamCikti alanlarını kullan: hesap vakalarında beklenen sayısal sonucu (1000/135/200) araç çıktısında ya da yanıt metninde ara; takvim-ekle'de hamGirdi'nin 14:00 ve 'toplantı', hatırlatıcıda 18:00 içerdiğini doğrula. Kod-asal'daki '1060' içerik kontrolünü (Degerlendirme.swift:80) diğer hesap/zaman/hatırlatıcı vakalarına genelle. Bu, ikon-only skorlamayı argüman-farkında hale getirir.

P2	Canlı format uyumunu (tablo satırı üretimi) bir iddiaya dönüştür	Zayıflık #5: talimatın en kırılğan format kuralı yalnız statik motor tarafında test ediliyor; modelin fiilen \	... \
P2	Zayıf iddiaları sıkılaştır ve ölü suSaat()'i bağla	Zayıflık #6: ":" gibi iddialar sahte-yeşil üretiyor ve ölü kod, test niyetinin paslandığını gösteriyor; ucuz ama gerçek kapsam kazandıran temizlik.	zaman-saat vakasını yanitIcermeli: suSaat() ile gerçek saat öneğine bağla (böylece ölü kod da devreye girer) ve genel olarak her yanitIcermeli iddiasını daha özgül yap. Bir iddianın 'her yanıtta bulunabilecek' bir alt-dizge olmadığını gözden geçir; aksi halde vaka koruma sağlamıyordur.